

運用生成式 AI 自動審查程式碼

來源：<https://codelabs.developers.google.com/genai-for-dev-code-review?hl=zh-tw>

步驟數：26

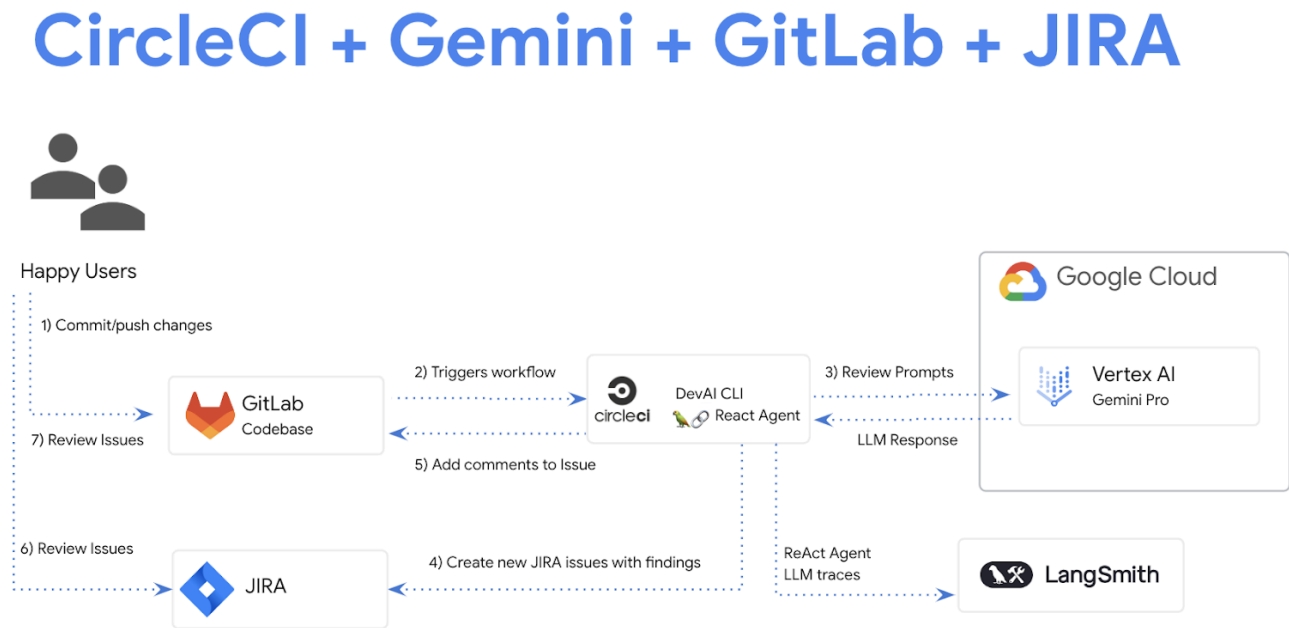
使用 GenAI 自動化程式碼審查

目錄

1. 總覽
2. 設定和需求
3. 測試提示的選項
4. 建立服務帳戶
5. 將 GitHub 存放區分支新增至個人 GitHub 存放區
6. 啟用 GitHub Actions 工作流程
7. 新增存放區密鑰
8. 執行 GitHub Actions 工作流程
9. 複製存放區
10. 啟用 Gemini Code Assist
11. 使用 Gemini Code Assist 解釋程式碼
12. 在本機執行 DevAI CLI
13. DevAI CLI 開發
14. 探索 devai CLI 指令
15. 在檔案中追蹤所有環境變數
16. LangSmith LLM 追蹤設定
17. JIRA 指令設定
18. GitLab 指令設定
19. DevAI CLI 開發
20. 將變更推送至 GitLab 存放區
21. GitLab CICD 設定
22. 執行 GitLab CICD 管道
23. 查看 GitLab 管道輸出內容
24. CircleCI 整合
25. 啟用 JIRA 和 GitLab 方法
26. 恭喜！

1. 總覽

在本實驗室中，您將設定 CI/CD 管道並整合 Gemini，自動執行程式碼審查步驟。



學習目標

在本實驗室中，您將瞭解如何執行下列操作：

- 如何在 GitHub、GitLab 和 CircleCI 中新增生成式 AI 程式碼審查自動化步驟
- 如何使用 LangChain ReAct 代理程式和工具包，自動執行在 GitLab 問題加上註解，以及開啟 JIRA 支援單等工作

必要條件

- 本實驗室假設您已熟悉 Cloud 控制台和 Cloud Shell 環境。

步驟 2 · 約 0 分鐘

2. 設定和需求

設定 Cloud 專案

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud console interface. At the top, there is a 'Select a project' dropdown menu. Below it, the 'Select a project' page is visible, featuring a search bar labeled 'Search projects and folders'. To the right of the search bar is a 'NEW PROJECT' button. Below the search bar, the 'New Project' form is shown. It includes a 'Project name' field with the value 'My Project 13475', a 'Project ID' field with the value 'inbound-analogy-389614' (highlighted with a blue box), and a 'Location' field. The 'Project ID' field has a note: 'It cannot be changed later. EDIT'. At the bottom of the form are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

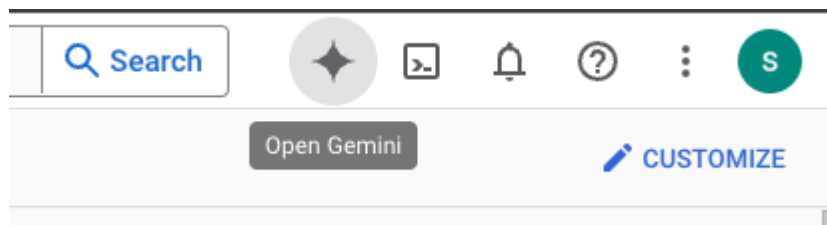
注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

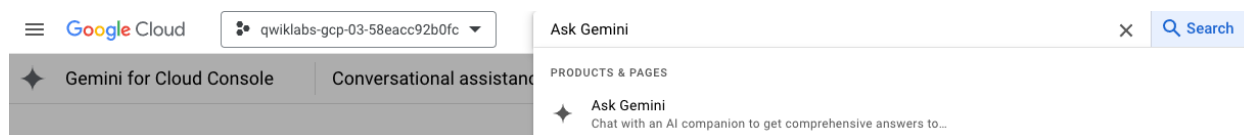
用者可參加[價值\\$300 美元的免費試用計畫](#)。

環境設定

開啟 Gemini 對話。



或在搜尋列中輸入「問問 Gemini」。



啟用 Cloud AI Companion API：

Required API

To use conversational assistance and other Gemini features, enable the Cloud AI Companion API in this project `qwiklabs-gcp-02-7035cd21994d` [Learn more](#)

[Gemini for Google Cloud](#)   Not enabled

[ENABLE](#) [SEND FEEDBACK](#)

Use of the Cloud AI Companion API is subject to the [Google Cloud Terms of Service](#) terms of service and [Service Specific Terms](#) applicable to Generative AI Services.

按一下「`Start chatting`」，然後按照其中一個範例問題操作，或自行輸入提示詞來試用。



Welcome to Gemini for Cloud console

Get help where you need it with a conversational assistant that answers your questions about Google Cloud, reviews code snippets, and troubleshoots issues quickly.

Gemini for Cloud console is a work in progress and may display inaccurate information. Your prompts aren't used to train the model. [Learn more](#)

Start chatting

[Learn more](#)

建議提示詞：

- 請用 5 個重點說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請用 5 個簡短重點向學生說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請用 5 個簡短重點向認證 Kubernetes 開發人員說明 Cloud Run。
- 您是 Google Cloud Run 產品經理，請以 5 個簡短重點，向資深開發人員說明何時該使用 Cloud Run，何時該使用 GKE。

如要進一步瞭解如何撰寫更優質的提示，請參閱[提示指南](#)。

瞭解 Gemini for Google Cloud 如何使用您的資料

Google 對隱私權的承諾

Google 是業界首批發布 [AI/機器學習隱私權承諾](#) 的公司，該文提到我們的信念：除了極致的安全性之外，客戶也應該對儲存在雲端的自家資料保有最大的掌控權。

您提交及收到的資料

您向 Gemini 提出的問題 (包括提交給 Gemini 分析或完成的任何輸入資訊或程式碼)，都稱為提示。您從 Gemini 取得的答案或程式碼完成建議稱為「回覆」。我們不會將您的提示詞或 Gemini 的回覆用做模型訓練資料。

提示加密

當您將提示提交給 Gemini 時，您的資料會在傳輸過程中加密，然後輸入 Gemini 的基礎模型。

Gemini 生成的節目資料

Gemini 是以 Google Cloud 第一方程式碼和精選第三方程式碼訓練而成。您必須對程式碼的安全性、測試和效用負責，包括 Gemini 提供的任何程式碼完成、生成或分析功能。

[進一步瞭解](#) Google 如何處理提示。

3. 測試提示的選項

如要變更/延長現有的 devai CLI 提示，有幾種做法可供選擇。

- [Vertex AI Studio](#)

Vertex AI Studio 是 Google Cloud Vertex AI 平台的一部分，專門用於簡化及加速生成式 AI 模型的開發和使用。

- [Google AI Studio](#)

Google AI Studio 是網頁工具，可讓您製作提示工程和 Gemini API 的原型並進行實驗。[註冊](#)使用脈絡窗口達 100 萬個詞元的 Gemini 1.5 Pro，或[進一步瞭解](#)。

- [Gemini 網頁應用程式](#) (gemini.google.com)

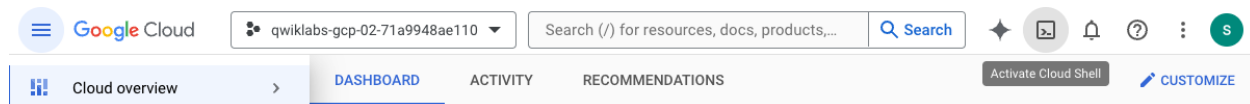
Google Gemini 網頁應用程式 (gemini.google.com) 是一款網頁工具，可協助您探索及運用 Google Gemini AI 模型強大的功能。

- [Android](#)和 [iOS 版 Google 應用程式](#)
-

步驟 4 · 約 0 分鐘

4. 建立服務帳戶

按一下搜尋列右側的圖示，啟用 Cloud Shell。



在開啟的終端機中，啟用使用 Vertex AI API 和 Gemini 對話 所需的服務。

```
gcloud services enable \  
  aiplatform.googleapis.com \  
  cloudaiaccompanion.googleapis.com \  
  cloudresourcemanager.googleapis.com \  
  secretmanager.googleapis.com
```

如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

執行下列指令，建立新的服務帳戶和金鑰。

您將使用這個服務帳戶，透過 CICD 管道對 Vertex AI Gemini API 發出 API 呼叫。

```
PROJECT_ID=$(gcloud config get-value project)
SERVICE_ACCOUNT_NAME='vertex-client'
DISPLAY_NAME='Vertex Client'
KEY_FILE_NAME='vertex-client-key'

gcloud iam service-accounts create $SERVICE_ACCOUNT_NAME --display-name "$DISPLAY_NAME"

gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/aiplatform.admin" --condition None

gcloud projects add-iam-policy-binding $PROJECT_ID --
member="serviceAccount:$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com" --
role="roles/secretmanager.secretAccessor" --condition None

gcloud iam service-accounts keys create $KEY_FILE_NAME.json --iam-
account=$SERVICE_ACCOUNT_NAME@$PROJECT_ID.iam.gserviceaccount.com
```

步驟 5 · 約 0 分鐘

5. 將 GitHub 存放區分支新增至個人 GitHub 存放區

前往 <https://github.com/GoogleCloudPlatform/genai-for-developers/fork>，然後選取您的 GitHub 使用者 ID 做為擁有者。

取消勾選只複製「主要」分支的選項。

請按一下 [`Create fork`]。

步驟 6 · 約 0 分鐘

6. 啟用 GitHub Actions 工作流程

在瀏覽器中開啟已分叉的 GitHub 存放區，然後切換至「Actions」分頁，啟用工作流程。



Workflows aren't being run on this forked repository

Because this repository contained workflow files when it was forked, we have disabled them from running on this fork. Make sure you understand the configured workflows and their expected usage before enabling Actions on this repository.

I understand my workflows, go ahead and enable them

[View the workflows directory](#)

步驟 7 · 約 0 分鐘

7. 新增存放區密鑰

在建立分支的 GitHub 存放區中，於「Settings / Secrets and variables / Actions」下方建立存放區密鑰。

新增名為「GOOGLE_API_CREDENTIALS」的存放區密鑰。

Repository secrets

New repository secret

Name 	Last updated
 GOOGLE_API_CREDENTIALS	last month  

切換至 Google Cloud Shell 視窗/分頁，並在 Cloud Shell 終端機中執行下列指令。

```
cat ~/vertex-client-key.json
```

複製檔案內容，並貼上做為密鑰的值。

Actions secrets / New secret

Name *

GOOGLE_API_CREDENTIALS

Secret *

```
{
  "client_email": "vertex-client@qwiklabs-gcp-03-6aeb0a5c4c76.iam.gserviceaccount.com",
  "client_id": "002454022266205162446",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
  "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/vertex-client%40qwiklabs-gcp-03-6aeb0a5c4c76.iam.gserviceaccount.com",
  "universe_domain": "googleapis.com"
}
```

Add secret

新增 PROJECT_ID 密鑰，並將 Qwiklabs 專案 ID 設為值

Actions secrets / New secret

Name *

PROJECT_ID

Secret *

gwiklabs-gcp-03-6aeb0a5c4c76

Add secret

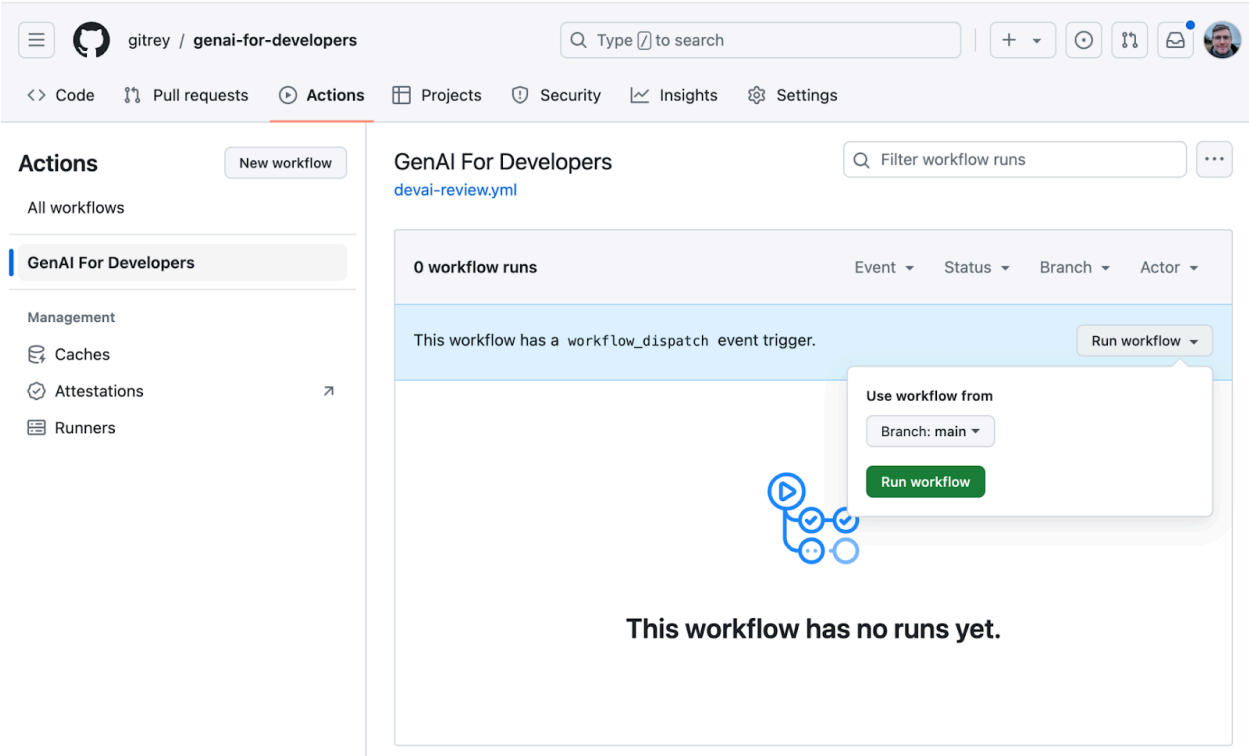
步驟 8 · 約 0 分鐘

8. 執行 GitHub Actions 工作流程

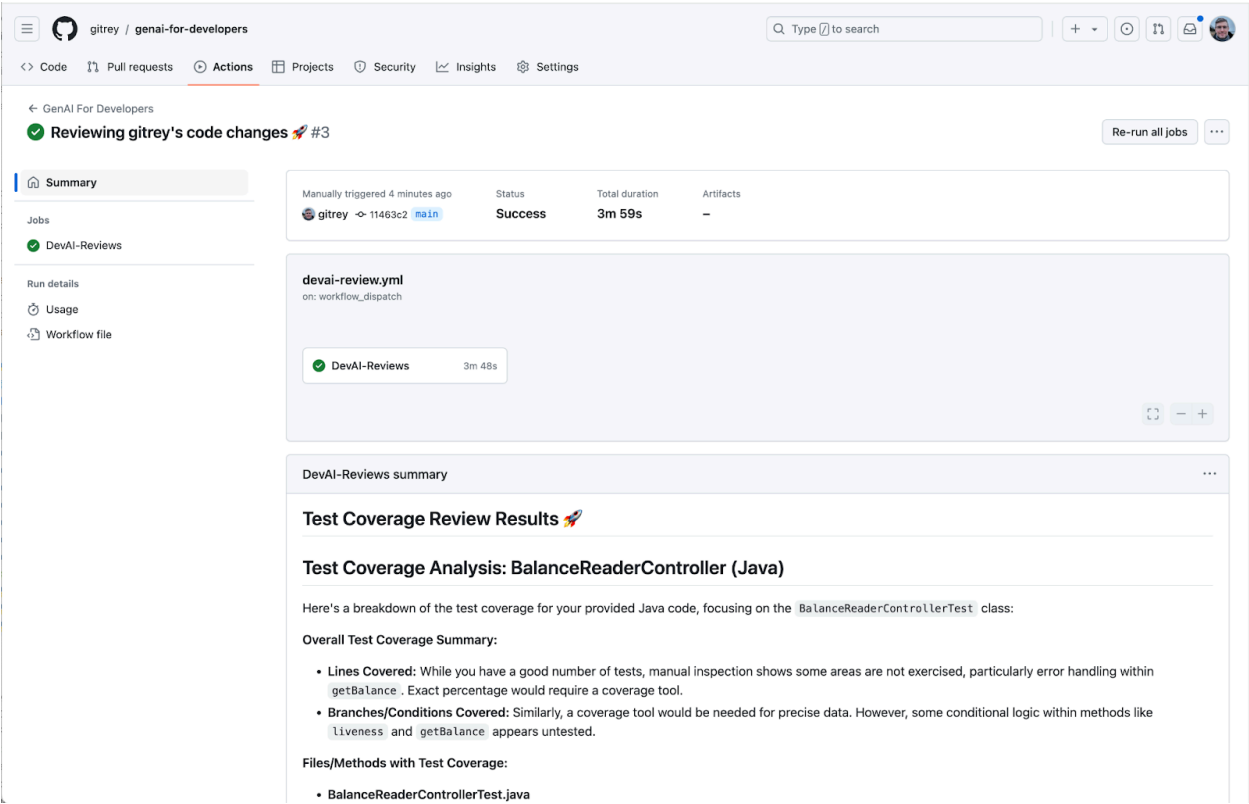
在瀏覽器中前往 GitHub 存放區，然後執行工作流程。

工作流程已設為在程式碼推送或手動執行時執行。

在「所有工作流程」下方選取「GenAI For Developers」，然後使用「main」分支點按「Run workflow」。



查看結果：



測試涵蓋範圍指令的結果：

```
devai review testcoverage -c ${github.workspace }/sample-app/src/main/java/anthos/samples/bankofanthos/balancereader
```

Summary

Jobs

DevAI-Reviews

Run details

Usage

Workflow file

Test Coverage Analysis: BalanceReaderController (Java)

Here's a breakdown of the test coverage for your provided Java code, focusing on the `BalanceReaderControllerTest` class:

Overall Test Coverage Summary:

- Lines Covered:** While you have a good number of tests, manual inspection shows some areas are not exercised, particularly error handling within `getBalance`. Exact percentage would require a coverage tool.
- Branches/Conditions Covered:** Similarly, a coverage tool would be needed for precise data. However, some conditional logic within methods like `liveness` and `getBalance` appears untested.

Files/Methods with Test Coverage:

- BalanceReaderControllerTest.java**
 - `version()` - Tests retrieving the version number.
 - `readiness()` - Tests the readiness endpoint.
 - `livenessSucceedsWhenLedgerReaderIsAlive()` - Tests liveness with a live LedgerReader.
 - `livenessFailsWhenLedgerReaderIsNotAlive()` - Tests liveness with a non-responsive LedgerReader.
 - `getBalanceSucceedsWhenAccountMatchesAuthenticatedUser()` - Tests successful balance retrieval.
 - `getBalanceIsCorrectWhenAccountMatchesAuthenticatedUser()` - Validates correct balance is returned.
 - `getBalanceFailsWhenAccountDoesNotMatchAuthenticatedUser()` - Tests unauthorized access attempt.
 - `getBalanceFailsWhenUserNotAuthenticated()` - Tests unauthenticated access attempt.
 - `getBalanceFailsWhenCacheThrowsError()` - Partially tests cache error handling (more on this below).

Files/Methods Lacking Test Coverage:

- BalanceReaderController.java**
 - Error handling within `getBalance`:** The code catches `ExecutionException` and `UncheckedExecutionException`, but the specific handling (beyond logging) isn't explicitly tested.

Recommendations for Improvement:

- Improve Error Handling Test Coverage in `getBalance`:**
 - Test specific error responses:** Create tests to simulate:
 - `ResourceAccessException` from `cache.get()` - This simulates the database being unavailable.
 - Test the logging mechanism within these error blocks to ensure it's working as intended.
 - Example:**

```
@Test
@DisplayName("Given the cache throws a ResourceAccessException, return 503")
void getBalanceReturnsServiceUnavailableWhenCacheThrowsResourceAccessException() throws Exception ***
// Given
when(verifier.verify(TOKEN)).thenReturn(jwt);
when(jwt.getClaim(JWT_ACCOUNT_KEY)).thenReturn(claim);
when(claim.asString()).thenReturn(AUTHED_ACCOUNT_NUM);
when(cache.get(AUTHED_ACCOUNT_NUM)).thenThrow(ResourceAccessException.class);

// When
final ResponseEntity actualResult = balanceReaderController.getBalance(BEARER_TOKEN, AUTHED_ACCOUNT_NUM);

// Then
assertNotNull(actualResult);
assertEquals(HttpStatus.SERVICE_UNAVAILABLE, actualResult.getStatusCode());
***
```
- Use a Coverage Tool:** Integrate a Java code coverage tool (like JaCoCo, Clover) into your build process. This will give you precise line and branch coverage metrics, helping you identify any untested areas systematically.
- Consider Additional Scenarios:**

程式碼審查指令的結果：

```
devai review code -c ${github.workspace }/sample-app/src/main/java/anthos/samples/bankofanthos/balancereader
```

Summary

Jobs

DevAI-Reviews

Run details

Usage

Workflow file

Code Review: Bank of Anthos - Balance Reader Service

This review analyzes the Balance Reader service for the Bank of Anthos application. It focuses on the provided code snippets and addresses potential improvements based on the areas you outlined: Correctness, Efficiency, Maintainability, Security, and Best Practices.

JWTVerifierGenerator:

- Security (High):**
 - Issue:** Storing the public key in a file (referenced by `PUB_KEY_PATH`) might be insecure, especially if the deployment environment doesn't protect file system access adequately.
 - Suggestion:**
 - Explore more secure ways to manage the public key, such as:
 - Using a Key Management Service (KMS) like HashiCorp Vault or AWS KMS for centralized key storage and retrieval.
 - If using Kubernetes, consider storing the public key as a Kubernetes Secret and mounting it securely into the application pod.
- Error Handling (Medium):**
 - Issue:** The `generateJWTVerifier` method catches broad exceptions (`IOException`, `NoSuchAlgorithmException`, `InvalidKeySpecException`) and wraps them in a custom `GenerateKeyException`. While this provides a consistent exception type, it might mask the underlying cause of the error, making debugging more difficult.
 - Suggestion:** Consider catching the exceptions individually and logging more specific error messages. You could rethrow the custom exception after logging to maintain the original behavior. For example:

```
try ***
// ...
*** catch (IOException e) ***
    LOGGER.error("Error reading public key file: **", e.getMessage());
    throw new GenerateKeyException("Cannot generate key: ", e);
*** catch (NoSuchAlgorithmException e) ***
    LOGGER.error("RSA algorithm not found: **", e.getMessage());
// ...
*** // ... other catch blocks
```
- Code Style (Low):**
 - Issue:** The logic for removing headers and whitespace from the public key string could be more readable.
 - Suggestion:** Consider using a regular expression to replace the multiple `replaceFirst` and `replaceAll` calls. Example:

```
keyStr = keyStr.replaceAll("(?m)^-----.*[^\s+\\s+$]", "");
```

This regex removes the "BEGIN PUBLIC KEY" and "END PUBLIC KEY" lines (with potential whitespace), leading and trailing whitespace from each line.

BalanceReaderController:

- Error Handling (Medium):**
 - Issue:** The `getBalance` method handles `ExecutionException` and `UncheckedExecutionException` from the cache with the same generic "cache error" message. This might not be informative enough for troubleshooting.
 - Suggestion:** Handle the exceptions separately and log more specific messages, indicating whether it was a problem with retrieving data from the cache or an error during the loading process.
- JWT Verification (Low):**
 - Issue:** The logic for extracting the token from the `Authorization` header could be made clearer.
 - Suggestion:** Consider using a regular expression or a dedicated library function for parsing the `Authorization` header to improve readability.
- Cache Management (Low):**
 - Question:** Have you considered using a time-based eviction policy for the cache (`expireAfterWrite`) in addition to the size-based eviction (`maximumSize`)? This could help to automatically refresh stale data from the database.

效能檢查指令的結果：

```
devai review performance -c ${github.workspace}}/sample-app/src/main/java/anthos/samples/bankofanthos/balancereader
```

Summary

Jobs

DevAI-Reviews

Run details

Usage

Workflow file

BalanceReader Application - Java Performance Review

Here's a performance review of the provided Java code, focusing on potential bottlenecks and optimization opportunities:

File: JWTVerifierGenerator.java

- Method: generateJWTVerifier()
 - Severity: Medium
 - Observation: The public key is loaded from the file system on each call to this method. This operation can be I/O intensive and may introduce latency, especially if the key is large or the file system is slow.
 - Suggestion: Consider caching the generated `JWTVerifier` instance. Since the public key is unlikely to change frequently, caching the verifier would eliminate redundant file I/O on each request. You could use a simple in-memory cache or explore more sophisticated caching mechanisms provided by Spring.

File: BalanceReaderController.java

- Method: getBalance()
 - Severity: Low
 - Observation: The `bearerToken` string manipulation (splitting on "Bearer ") could be slightly optimized.
 - Suggestion: Consider using `startsWith` and `substring` instead of `split` for a minor performance gain:

```
if (bearerToken.startsWith("Bearer ")) ***
    bearerToken = bearerToken.substring(7); // "Bearer " has length 7
***
```

File: BalanceCache.java

- Observation: The cache implementation utilizes a `LoadingCache` with a specified maximum size. This is a good practice to prevent unbounded cache growth.
- Suggestion (Open Question):
 - What is the expected hit rate for this cache?
 - If the hit rate is low, you might consider using a cache with eviction policies based on time or usage patterns to prevent stale data.

File: LedgerReader.java

- Observation: The `LedgerReader` employs a polling mechanism to check for new transactions.
- Suggestion (Open Question):
 - Would a more event-driven approach be more efficient than polling? For example, could the ledger database emit events when new transactions are added?
 - This would eliminate the need for polling and potentially reduce latency in processing new transactions.

General Observations and Suggestions:

- Logging: The code includes logging statements. While logging is essential for debugging and monitoring, ensure that excessive logging, especially at debug levels, doesn't impact performance in production. Consider using asynchronous logging frameworks or configuring log levels appropriately.
- Database Interaction: The code interacts with a database (`TransactionRepository`). Analyze database queries for potential optimization:
 - Ensure that appropriate indexes are defined on the `TRANSACTIONS` table to speed up queries.
 - Investigate if the native query used in `TransactionRepository.findBalance` can be optimized or replaced with a more efficient JPQL query.
- Error Handling: The code includes error handling with appropriate HTTP status codes. Ensure that exceptions are handled efficiently and don't introduce unnecessary overhead.

No Major Issues Found:

Overall, the code demonstrates good coding practices and a solid understanding of performance considerations. The suggestions provided aim to address potential areas for improvement and encourage further exploration based on the application's specific context and usage patterns.

安全審查指令的結果：

```
devai review security -c ${github.workspace }/sample-app/src/main/java/anthos/samples/bankofanthos/balancereader
```

Summary

Jobs

DevAI-Reviews

Run details

Usage

Workflow file

Security Review Results 🚀

Balance Reader Code Review - Java

This code implements a balance reader service that tracks bank balances. It uses JWT for authentication and authorization, a cache for performance, and a background thread to poll for new transactions.

Here's a security review based on the provided code and common Java security considerations:

File: JWTVerifierGenerator.java

Function: generateJWTVerifier

Issue: Potential exposure of private key details in logs

- **Type of vulnerability:** Information Disclosure
- **Location:**
 - `LOGGER.error(String.format("Failed initializing JWT verifier: %s", e.toString()));`
- **Potential impact:** Exception stack traces could reveal sensitive information about the private key, keystore, or file system paths if an error occurs during key loading.
- **Severity:** Medium
- **Recommendation:**
 - Avoid logging the raw exception using `e.toString()`.
 - Instead, log a generic error message and log the exception itself using the appropriate logging framework methods (e.g., `LOGGER.error("Failed initializing JWT verifier", e);`). This allows for more controlled logging without exposing sensitive data.

File: BalanceReaderController.java

Function: getBalance

Issue 1: Missing Input Validation for "accountId"

- **Type of vulnerability:** Potential for IDOR (Insecure Direct Object Reference) or unexpected behavior
- **Location:**
 - `@PathVariable String accountId`
- **Potential impact:** An attacker might be able to manipulate the `accountId` parameter to access balances of other accounts if no validation is performed to ensure it matches a format or pattern associated with valid account IDs.
- **Severity:** High
- **Recommendation:**
 - Validate the `accountId` input before using it. Check if it conforms to expected format (e.g., length restrictions, alphanumeric characters only) using regular expressions or dedicated validation libraries.

Issue 2: Incomplete Authorization Check

- **Type of vulnerability:** Authorization Bypass
- **Location:**
 - `if (!accountId.equals(jwt.getClaim("acct").asString())) *** ... ***`
- **Potential impact:** The authorization check only compares the `accountId` from the request with the "acct" claim in the JWT. If the JWT verification logic does not strictly enforce the presence and format of the "acct" claim, an attacker might be able to craft a valid JWT without the "acct" claim, bypassing the authorization check.
- **Severity:** High
- **Recommendation:**
 - Enforce the presence and validation of the "acct" claim during JWT verification.
 - If the "acct" claim is missing or has an unexpected format, reject the JWT as invalid.

General Observations:

- **Session Management:** The code doesn't explicitly manage sessions. If session management is required, ensure secure cookie settings and implement measures to prevent session fixation and hijacking.

檢視阻斷程式審查指令的結果：

```
devai review blockers -c ${github.workspace} /sample-app/pom.xml
```

Summary

Jobs

DevAI-Reviews

Run details

Usage

Workflow file

Blockers Review Results 🚀

```
***
"onboarding_status": "BLOCKED",
"blockers": ["IBM MQ"]
***
```

Explanation:

The provided code includes the following dependency, which signifies the usage of IBM MQ:

```
<dependency>
  <groupId>com.ibm.mq</groupId>
  <artifactId>com.ibm.mq.allclient</artifactId>
  <version>9.2.2.0</version>
</dependency>
```

Since "IBM MQ" is listed as a blocker, the onboarding status is set to "BLOCKED".

[Job summary generated at run-time](#)

步驟 9 · 約 0 分鐘

9. 複製存放區

返回 Cloud Shell 終端機，然後複製存放區。

為 GitHub 存放區建立資料夾。

```
mkdir github
cd github
```

執行指令前，請將 `YOUR-GITHUB-USERID` 變更為 GitHub 使用者 ID。

在終端機中設定 Git 使用者名稱和電子郵件地址。

請先更新值，再執行指令。

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

```
git clone https://github.com/YOUR-GITHUB-USERID/genai-for-developers.git
```

變更資料夾，並在 Cloud Shell 編輯器中開啟工作流程檔案。

```
cd genai-for-developers

cloudshell edit .github/workflows/devai-review.yml
```

等待 IDE 顯示設定檔。

The screenshot displays the Google Cloud Shell interface. At the top, the Google Cloud logo and project ID 'qwiklabs-gcp-00-5d4f8ee7057b' are visible. Below the header, the 'CLOUD SHELL Editor' window is open. The left sidebar shows the 'EXPLORER' view with a file tree for the 'STUDENT_00_F2E168A94E71' project. The tree includes a folder 'genai-for-developers' which contains a subfolder '.github/workflows' and a file 'devai-review.yml'. The main editor area shows the content of 'devai-review.yml', which is a GitHub Actions workflow for AI-enabled code reviews. The workflow includes steps for checking out code, listing files, setting up Python, and installing dependencies. The workflow is triggered on push or workflow_dispatch.

```
genai-for-developers > .github > workflows > ! devai-review.yml > {} jobs > {} DevAI-Reviews > [ ] steps > {} with
1  # AI enabled code reviews using devai
2  # - Requires GCP service account stored in GOOGLE_API_CREDENTIALS secret
3  name: GenAI For Developers
4  run-name: Reviewing ${github.actor}'s code changes 🚀
5  on: [push, workflow_dispatch]
6  env:
7    LOCATION: us-central1
8    PROJECT_ID: '${secrets.PROJECT_ID}'
9  jobs:
10   DevAI-Reviews:
11     runs-on: ubuntu-latest
12     steps:
13       - name: Check out repository code
14         uses: actions/checkout@v4
15       - name: List files in the repository
16         run: |
17           ls ${github.workspace}
18       - name: Set up Python
19         uses: actions/setup-python@v5
20         with:
21           python-version: '3.11.7'
22       - run: cd ${github.workspace}/devai-cli
23       - run: pip install virtualenv
24       - run: virtualenv venv
25       - run: source venv/bin/activate
26       - run: pip install -r ${github.workspace}/devai-cli/src/requirements.txt
27       - run: pip install --editable ${github.workspace}/devai-cli/src
```

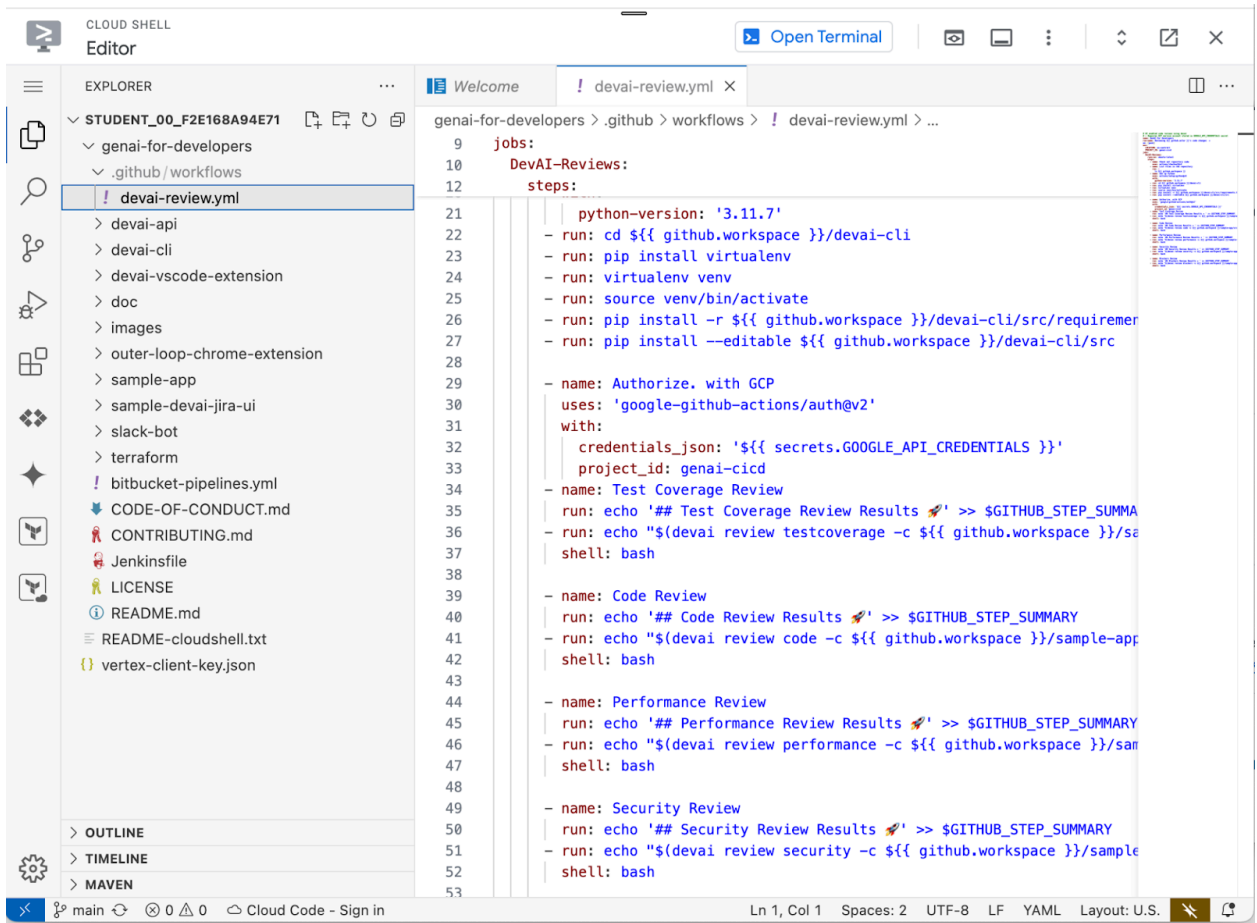

步驟 10 · 約 0 分鐘

10. 啟用 Gemini Code Assist

按一下右下角的「Gemini」圖示



按一下「Login to Google Cloud」和「Select a Google Cloud Project」。





CLOUD SHELL

Editor



GEMINI CODE ASSIST: CHAT



Welcome

! devai-review.yml X



Welcome to Gemini Code Assist

Supercharge your workflow with AI
powered chat, code completion,
code generation, and more.

[Login to Google Cloud](#)

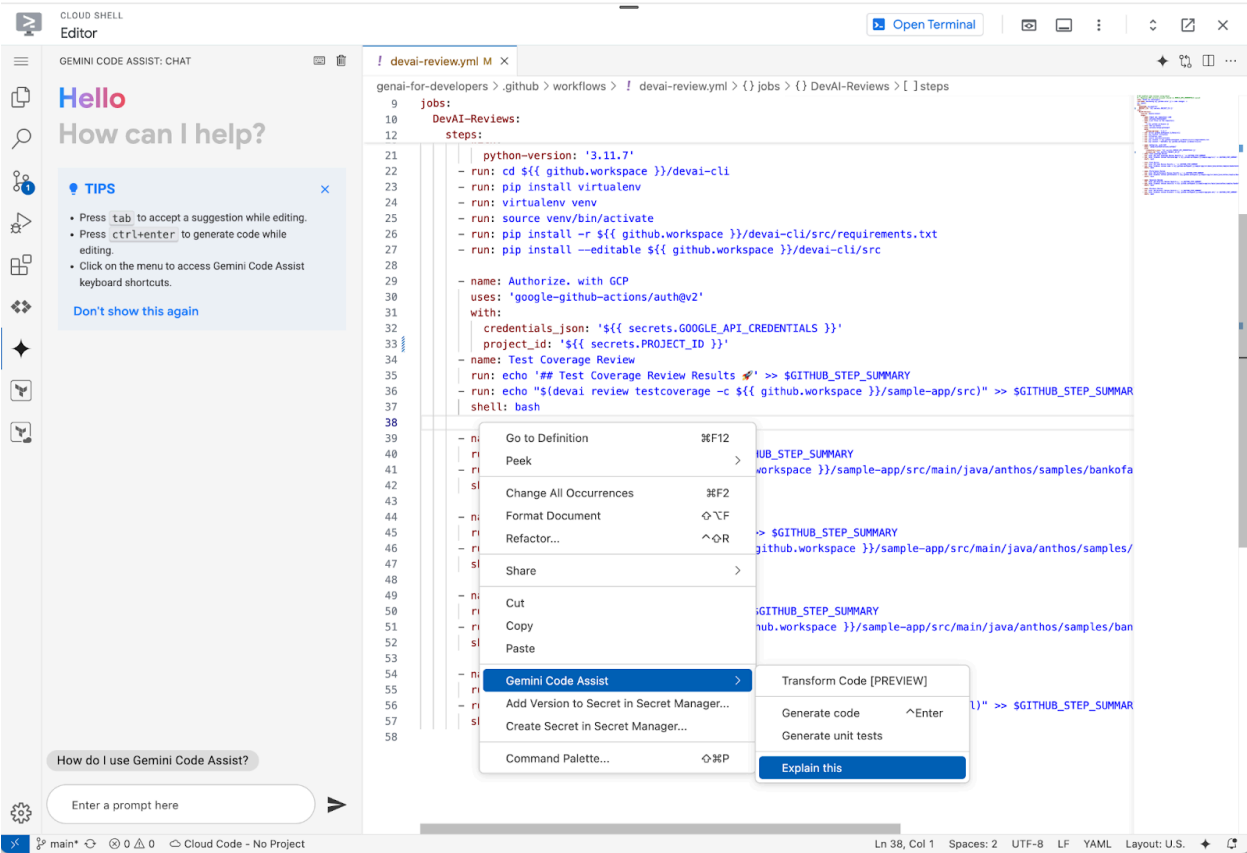
```
genai-for-developers > .github > workflows >
1  # AI enabled code reviews using
2  # - Requires GCP service account
3  name: GenAI For Developers
4  run-name: Reviewing ${ github.
5  on: [push, workflow_dispatch]
6  env:
7    LOCATION: us-central1
8    PROJECT_ID: '${ secrets.PROJ
9  jobs:
10   DevAI-Reviews:
11     runs-on: ubuntu-latest
12     steps:
13       - name: Check out reposit
14         uses: actions/checkout@
15       - name: List files in the
16         run: |
17           ls ${ github.workspac
18       - name: Set up Python
19         uses: actions/setup-pyth
20         with:
21           python-version: '3.11.
22       - run: cd ${ github.works
23       - run: pip install virtual
24       - run: virtualenv venv
25       - run: source venv/bin/act
26       - run: pip install -r ${
27       - run: pip install --edit
28
29       - name: Authorize. with GC
30         uses: 'google-github-act
31         with:
32           credentials_json: '${
33           project_id: '${ secre
34       - name: Test Coverage Revi
35         run: echo '## Test Cover
36       - run: echo "${devai revie
37         shell: bash
38
39       - name: Code Review
40         run: echo '## Code Revie
41       - run: echo "${devai revie
42         shell: bash
43
```

在彈出式視窗中，選取 Qwiklabs 專案。

The screenshot shows the Google Cloud Shell Editor interface. The top bar includes the 'CLOUD SHELL Editor' title and an 'Open Terminal' button. Below the top bar, there's a navigation bar with icons for file explorer, search, and other tools. The main workspace is divided into three panes. The left pane contains a 'Hello' message and instructions to select a Google Cloud project. The middle pane shows a terminal window with a list of Google Cloud projects, including 'qwiklabs-gcp-03-6aeb0a5c4c76'. The right pane shows a terminal window with a script for setting up the development environment, including steps for installing virtualenv, setting up the environment, and running tests.

11. 使用 Gemini Code Assist 解釋程式碼

在 `devai-review.yml` 檔案的任意處按一下滑鼠右鍵，然後選取 `Gemini Code Assist > Explain`。



審查說明：



GEMINI CODE ASSIST: CHAT



- `PROJECT_ID: '${{ secrets.PROJECT_ID }}'` - Retrieves the Google Cloud project ID from a secret named `PROJECT_ID`.

Job: DevAI-Reviews

- `runs-on: ubuntu-latest` - The job runs on a standard Ubuntu virtual machine.

Steps:**1. Check out repository code:**

- `uses: actions/checkout@v4` - Downloads the repository's code to the virtual machine.

2. List files in the repository:

- `run: ls ${{ github.workspace }}` - Lists the files in the repository's working directory.

3. Set up Python:

- `uses: actions/setup-python@v5` - Installs Python version 3.11.7.

4. Install DevAI CLI:

- `cd ${{ github.workspace }}/devai-cli` - Navigates to the DevAI CLI directory.
- `pip install virtualenv` - Installs the `virtualenv` package for creating isolated Python environments.
- `virtualenv venv` - Creates a virtual environment named `venv`.
- `source venv/bin/activate` - Activates the virtual environment.
- `pip install -r ${{ github.workspace }}/devai-cli/src/requirements.txt` - Installs dependencies from the `requirements.txt` file.
- `pip install --editable ${{ github.workspace }}/devai-cli/src` - Installs the DevAI CLI in editable mode.

5. Authorize with GCP:

- `uses: 'google-github-actions/auth@v2'` - Uses the Google Cloud authentication action.
- `credentials_json: '${{ secrets.GOOGLE_API_CREDENTIALS }}'` - Retrieves the Google Cloud service account credentials from a secret named `GOOGLE_API_CREDENTIALS`.
- `project_id: '${{ secrets.PROJECT_ID }}'` - Specifies the Google Cloud project ID.

6. Run DevAI Reviews:

- **Test Coverage Review:**
 - `devai review testcoverage -c ${{ github.workspace }}/sample-app/src` - Runs the DevAI test coverage review on the code in the `sample-app/src` directory.
- **Code Review:**
 - `devai review code -c ${{ github.workspace }}/sample-app/src/main/java/anthos/samples/bankofanthos/balancereader` - Runs the DevAI code review on the specified Java code.
- **Performance Review:**
 - `devai review performance -c ${{ github.workspace }}/sample-app/src/main/java/anthos/samples/bankofanthos/balancereader` - Runs the DevAI performance review on the specified Java code.
- **Security Review:**
 - `devai review security -c ${{ github.workspace }}/sample-app/src/main/java/anthos/samples/bankofanthos/balancereader` - Runs the DevAI security review on the specified Java code.
- **Blockers Review:**
 - `devai review blockers -c ${{ github.workspace }}/sample-app/pom.xml` - Runs the DevAI blockers review on the `pom.xml` file.

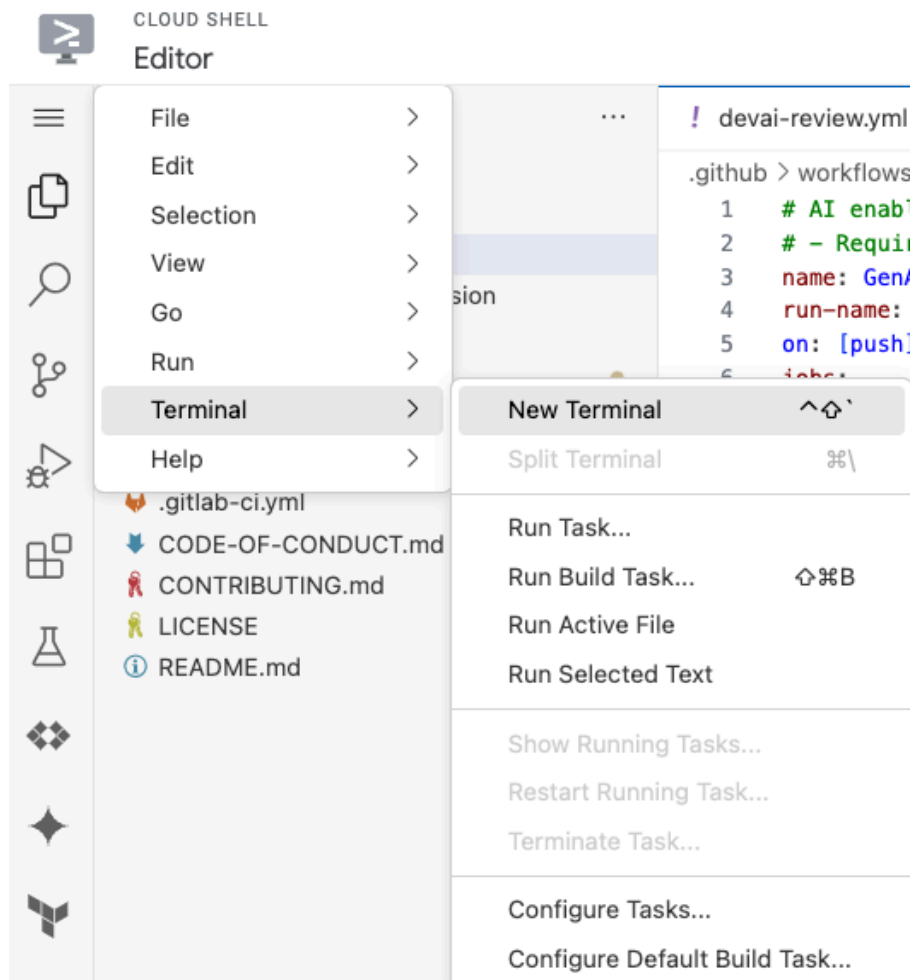
Summary:

This workflow automates code reviews using DevAI, providing insights into test coverage, code quality, performance, security, and potential blockers. It leverages Google Cloud's authentication and DevAI service to perform these reviews. The results are then displayed in the GitHub Actions workflow run summary.

步驟 12 · 約 0 分鐘

12. 在本機執行 DEVAI CLI

返回 Cloud Shell 編輯器，然後開啟新的終端機。



返回 Cloud Shell 終端機，然後執行下列指令，在本機安裝 `devai`。

```
pip3 install devai-cli
```

已安裝 CLI，但不在 PATH 中。

```
WARNING: The script devai is installed in '/home/student_00_478dfef8df15/.local/bin'
which is not on PATH.
```

```
Consider adding this directory to PATH or, if you prefer to suppress this warning,
use --no-warn-script-location.
```

執行下列指令，更新 PATH 環境變數。並替換成使用者的主資料夾名稱。例如：`student_00_478dfef8df15`

```
export PATH=$PATH:/home/YOUR-USER-HOME-FOLDER/.local/bin
```

執行 `devai CLI` 指令，在本機執行程式碼審查。查看 CLI 輸出內容。

```
export PROJECT_ID=$(gcloud config get-value project)
export LOCATION=us-central1
```

```
cd ~/github/genai-for-developers
```

```
devai review code -c ./sample-app/src/main/java/anthos/samples/bankofanthos/balancereader
```

執行下列指令，開啟檢視指令碼：

```
cloudshell edit devai-cli/src/devai/commands/review.py
```

在 `review.py` 檔案的任意處按一下滑鼠右鍵，然後選取 **Gemini Code Assist > Explain**。

查看說明。

The screenshot displays the Cloud Shell Editor interface. On the left, the Gemini Code Assist chat window is open, showing an 'Explain this' prompt. The chat response provides a detailed breakdown of the `review.py` code, organized into sections: 1. Imports and Setup, 2. Secret Management, 3. Image Handling, 4. Code Review Commands, and 5. Main Group. It explains the purpose of various imports, functions like `get_prompt` and `load_image_from_path`, and the logic of the `review` command. The right pane shows the `review.py` code file, which includes a copyright notice for Google LLC, imports for `click`, `devai.util.file_processor`, `format_files_as_string`, `vertexai.generative_models`, `google.cloud.aiplatform`, `telemetry`, `os`, `google.cloud.secretmanager`, `google.api_core.exceptions`, `google.api_core.gapic_v1.client_info`, and `logging`. The code defines a `USER_AGENT`, a `model_name`, and a `ensure_env_variable` function. The `get_prompt` function is defined to retrieve a secret from Google Secret Manager. The `review` command is implemented as a `click` command that takes a code path as input and performs a code review using the Gemini API.

步驟 13 · 約 0 分鐘

13. DevAI CLI 開發

在本節中，您將變更 devai CLI。

首先，請設定 Python 虛擬環境、安裝必要條件，然後執行範例指令。

```
cd ~/github/genai-for-developers/devai-cli
python3 -m venv venv
. venv/bin/activate
pip3 install -r src/requirements.txt
pip3 install --editable ./src
devai echo
```

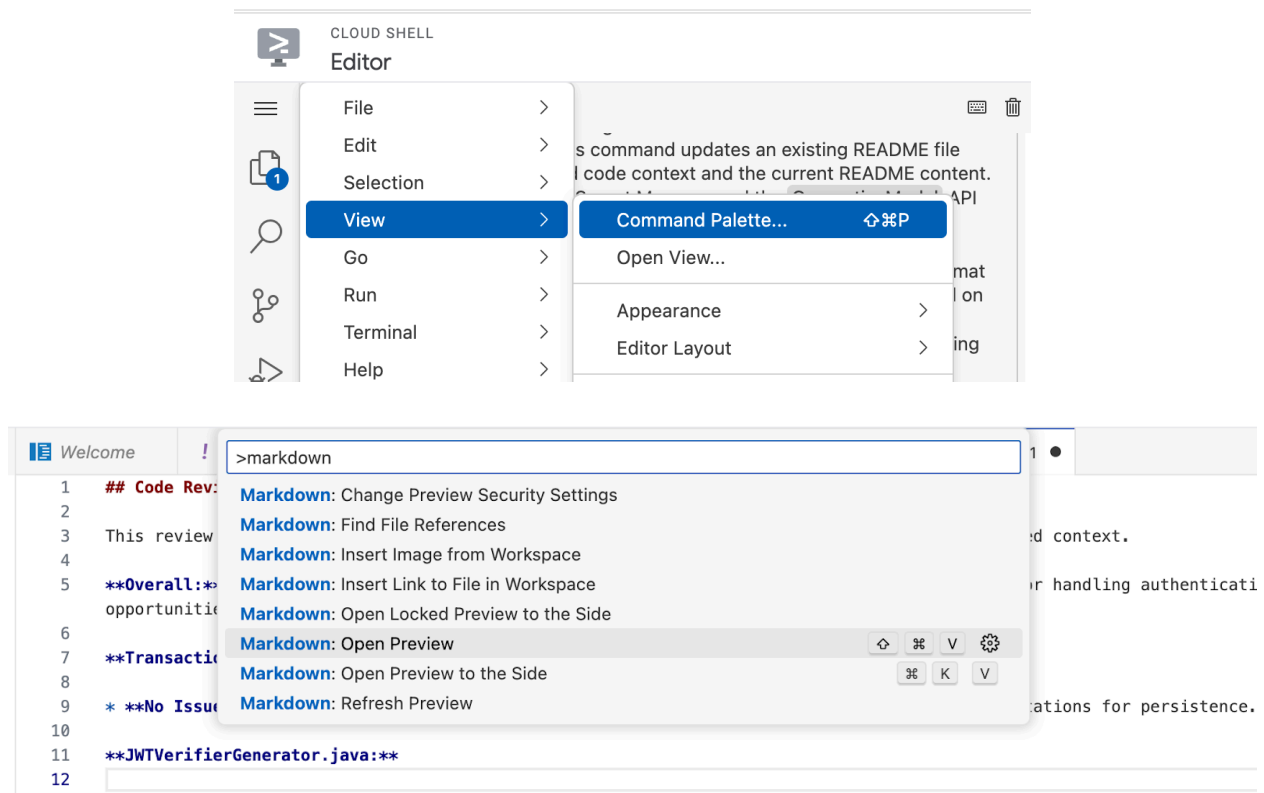
執行測試涵蓋範圍檢查指令，確認一切運作正常：

```
devai review testcoverage -c ~/github/genai-for-developers/sample-app/src
```

在 Cloud Shell 編輯器中使用 Markdown 預覽功能查看結果。

建立新檔案，然後貼上 Gemini 的回覆。

然後使用指令區塊面板，選取「Markdown: Open Preview」。



Test Coverage Analysis: BalanceReaderController (Java)

This analysis focuses on the Java code you provided for the `BalanceReaderController` and its associated test suite.

Overall Test Coverage Summary:

While you have a good start on testing, there are opportunities for more comprehensive coverage.

- Lines Covered:** Without a coverage report, I can't give an exact percentage. Visually, it seems many lines are covered, but some critical areas are missing.
- Branches/Conditions Covered:** Again, without a report, it's difficult to assess. However, the existing tests primarily focus on happy path scenarios, leaving conditional logic and edge cases potentially untested.

Files/Methods with Test Coverage:

- File:** `/home/student_00_f2e168a94e71/github/genai-for-developers/sample-app/src/test/java/anthos/samples/bankofanthos/balancereader/BalanceReaderControllerTest.java`
 - Method:** `version()` - Tested for successful response with version number.
 - Method:** `readiness()` - Tested for successful "ok" response.
 - Method:** `livenessSucceedsWhenLedgerReaderIsAlive()` - Tests success when `LedgerReader` is alive.
 - Method:** `livenessFailsWhenLedgerReaderIsNotAlive()` - Tests failure when `LedgerReader` is not alive.
 - Method:** `getBalanceSucceedsWhenAccountMatchesAuthenticatedUser()` - Tests success with matching authenticated user.
 - Method:** `getBalanceIsCorrectWhenAccountMatchesAuthenticatedUser()` - Tests for correct balance retrieval.
 - Method:** `getBalanceFailsWhenAccountDoesNotMatchAuthenticatedUser()` - Tests unauthorized access.
 - Method:** `getBalanceFailsWhenUserNotAuthenticated()` - Tests unauthorized access (no authentication).
 - Method:** `getBalanceFailsWhenCacheThrowsError()` - Tests error handling when the cache throws an exception.

Files/Methods Lacking Test Coverage:

- File:** `/home/student_00_f2e168a94e71/github/genai-for-developers/sample-app/src/main/java/anthos/samples/bankofanthos/balancereader/BalanceReaderController.java`
 - Method:** Constructor `(BalanceReaderController(LedgerReader, JWTVerifier, StackdriverMeterRegistry, LoadingCache, String, String))` - **High Priority**
 - Reason:** Contains critical setup logic for the `LedgerReader`, cache, and JWT verifier.
 - Test Scenarios:**
 - Verify the `LedgerReader` is started correctly and its callback is set.
 - Ensure the cache is initialized with the provided `meterRegistry`.
 - Method:** The callback logic within the constructor (lambda passed to `LedgerReader.startWithCallback`) - **High Priority**
 - Reason:** This logic updates the cache based on transactions, making it central to the application's core functionality.
 - Test Scenarios:**
 - Simulate incoming transactions and verify the cache is updated correctly for both debit and credit scenarios. Consider using a mock `LedgerReader` or directly invoking the callback logic.

步驟 14 · 約 0 分鐘

14. 探索 devai CLI 指令

程式碼審查指令

```
devai review code -c ~/github/genai-for-developers/sample-app/src/main/java
```

成效回顧指令

```
devai review performance -c ~/github/genai-for-developers/sample-app/src/main/java
```

安全審查指令

```
devai review security -c ~/github/genai-for-developers/sample-app/src/main/java
```

測試涵蓋範圍檢查指令

```
devai review testcoverage -c ~/github/genai-for-developers/sample-app/src
```

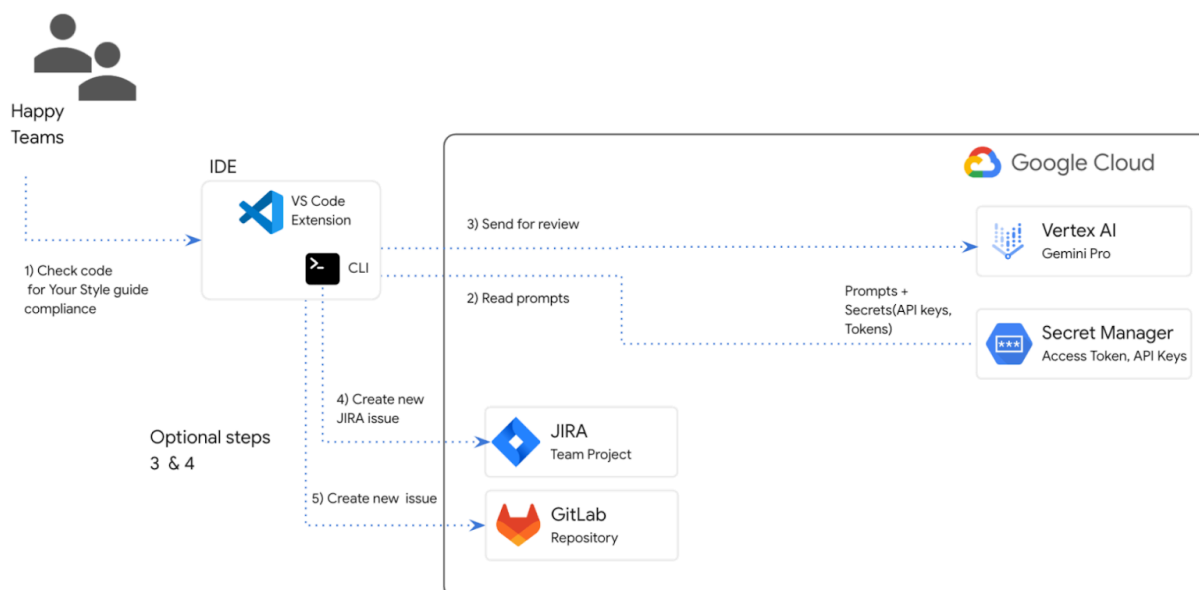
封鎖者審查指令

```
devai review blockers -c ~/github/genai-for-developers/sample-app/pom.xml  
devai review blockers -c ~/github/genai-for-developers/sample-app/setup.md
```

圖片/圖表審查和摘要：

輸入圖表 [~/github/genai-for-developers/images/extension-diagram.png]：

VS Code extension or CLI to perform code review locally



查看指令：

```
devai review image \  
-f ~/github/genai-for-developers/images/extension-diagram.png \  
-p "Review and summarize this diagram"
```

輸出內容：

The diagram outlines a process for conducting local code reviews using a VS Code extension or CLI, leveraging Google Cloud's Vertex AI (Gemini Pro) for generating review prompts.

****Process Flow:****

1. ****Code Style Check:**** Developers initiate the process by checking their code for adherence to pre-defined style guidelines.
2. ****Prompt Generation:**** The VS Code extension/CLI sends the code to Vertex AI (Gemini Pro) on Google Cloud.
3. ****Vertex AI Review:**** Vertex AI analyzes the code and generates relevant review prompts.
4. ****Local Review:**** The prompts are sent back to the developer's IDE for their consideration.
5. ****Optional Actions:**** Developers can optionally:
 - Create new JIRA issues directly from the IDE based on the review prompts.
 - Generate new issues in a GitLab repository.

****Key Components:****

- * ****VS Code Extension/CLI:**** Tools facilitating the interaction with Vertex AI and potential integrations with JIRA and GitLab.
- * ****Vertex AI (Gemini Pro):**** Google Cloud's generative AI service responsible for understanding the code and generating meaningful review prompts.
- * ****Google Cloud Secret Manager:**** Securely stores API keys and access tokens required to authenticate and interact with Google Cloud services.
- * ****JIRA/GitLab (Optional):**** Issue tracking and project management tools that can be integrated for a streamlined workflow.

****Benefits:****

- * ****Automated Review Assistance:**** Leveraging AI to generate review prompts saves time and improves the consistency and quality of code reviews.
- * ****Local Development:**** The process empowers developers to conduct reviews locally within their familiar IDE.
- * ****Integration Options:**** The flexibility to integrate with project management tools like JIRA and GitLab streamlines workflow and issue tracking.

圖片差異分析：

```
devai review imgdiff \  
-c ~/github/genai-for-developers/images/devai-api.png \  
-t ~/github/genai-for-developers/images/devai-api-slack.png
```

輸出内容：

The following UI elements are missing in the "AFTER UPGRADE STATE" image compared to the "BEFORE UPGRADE STATE" image:

1. ****Slack:**** The entire Slack element, including the icon, "Team channel" label, and the arrow indicating interaction, is absent in the AFTER UPGRADE image.
2. ****Storage Bucket:**** The "Storage Bucket" element with its icon and "PDFs" label is missing in the AFTER UPGRADE image.
3. ****"GenAI Agents" label in Vertex AI block:**** The BEFORE UPGRADE image has "Vertex AI Agents" and "GenAI Agent" labels within the Vertex AI block, while the AFTER UPGRADE image only has "Vertex AI."
4. ****"Open JIRA Issue" and "Team Project" labels:**** In the BEFORE UPGRADE image, these labels are connected to the JIRA block with an arrow. These are missing in the AFTER UPGRADE image.

****Decision Explanation:****

The analysis is based on a direct visual comparison of the two provided images, noting the presence and absence of specific UI elements and their associated labels. The elements listed above are present in the BEFORE UPGRADE image but absent in the AFTER UPGRADE image.

文件生成指令：

```
devai document readme -c ~/github/genai-for-developers/sample-app/src/main/
```

輸出内容：

Bank of Anthos - Balance Reader Service

Table of Contents

- [Description](#description)
- [Features](#features)
- [Technologies Used](#technologies-used)
- [Installation](#installation)
- [Configuration](#configuration)
- [Usage](#usage)
- [Health Checks](#health-checks)
- [Metrics and Tracing](#metrics-and-tracing)
- [Contributing](#contributing)
- [License](#license)

Description

The Balance Reader service is a component of the Bank of Anthos sample application. It provides a REST endpoint for retrieving the current balance of a user account. This service demonstrates key concepts for building microservices with Spring Boot and deploying them to a Kubernetes cluster.

Features

- Securely retrieves account balances using JWT authentication.
- Leverages a local cache for fast balance retrieval.
- Asynchronously processes transactions from a central ledger.
- Provides health check endpoints for Kubernetes liveness and readiness probes.
- Exposes metrics to Stackdriver for monitoring and observability.
- Supports distributed tracing with Zipkin.

Technologies Used

- Java
- Spring Boot
- Spring Data JPA
- Hibernate
- Google Cloud SQL (PostgreSQL)
- JWT (JSON Web Token)
- Guava Cache
- Micrometer
- Stackdriver
- Zipkin

Installation

1. ****Prerequisites:****
 - Java 17 or later
 - Maven 3.5 or later

- Docker (for containerization)
- Kubernetes cluster (for deployment)
- Google Cloud account (for Stackdriver and other GCP services)

在 Cloud Shell 編輯器中查看可用的 devai 指令列指令：

```
cloudshell edit ~/github/genai-for-developers/devai-cli/README.md
```

或查看 GitHub 存放區中的 [README.md](#)。

步驟 15 · 約 0 分鐘

15. 在檔案中追蹤所有環境變數

建立新檔案，追蹤您建立的所有環境變數 (例如 API 金鑰、API 權杖等)。

您會在實驗室中多次使用這些值，因此集中管理可方便參考。

步驟 16 · 約 0 分鐘

16. LangSmith LLM 追蹤設定

建立 LangSmith 帳戶，並在「設定」部分產生服務 API 金鑰。 <https://docs.smith.langchain.com/>

設定 LangSmith 整合所需的環境變數。請先替換服務 API 金鑰，再執行指令。

```
export LANGCHAIN_API_KEY=langsmith-service-api-key

export LANGCHAIN_TRACING_V2=true
export LANGCHAIN_ENDPOINT="https://api.smith.langchain.com"
```

為避免在終端機中公開私密資訊，最佳做法是使用 `read -s`，這是設定環境變數的安全方式，不會在控制台的指令記錄中顯示值。執行後，您必須貼上該值並按下 Enter 鍵。

步驟 17 · 約 0 分鐘

17. JIRA 指令設定

如果沒有 JIRA 帳戶，請[建立帳戶](#)。

為專案建立 JIRA API 權杖。 <https://id.atlassian.com/manage-profile/security/api-tokens>

設定 JIRA 整合所需的環境變數 (請先替換值，再執行指令)。

```
export JIRA_API_TOKEN=your-token-value
export JIRA_USERNAME="email that you used to register with JIRA"
export JIRA_INSTANCE_URL="https://YOUR-PROJECT.atlassian.net"
export JIRA_PROJECT_KEY="JIRA project key"
export JIRA_CLOUD=true
```

開啟 `review.py` 檔案：

```
cloudshell edit ~/github/genai-for-developers/devai-cli/src/devai/commands/review.py
```

查看 `review.py` 個檔案：

```
source=source.format(format_files_as_string(context))

code_chat_model = GenerativeModel(model_name)
code_chat = code_chat_model.start_chat()
code_chat.send_message(qry)
response = code_chat.send_message(source)
...
else:
    click.echo(response.text)
```

找出並取消註解下方這一行：

```
# Uncomment after configuring JIRA and GitLab env variables - see README.md for
details
```

在檔案頂端匯入 JIRA 指令

```
# from devai.commands.jira import create_jira_issue
```

在 `code` 方法中建立 JIRA 問題的方法

```
#create_jira_issue("Code Review Results", response.text)
```

重新執行程式碼審查指令，並檢查代理程式的輸出內容：

```
export PROJECT_ID=$(gcloud config get-value project)
export LOCATION=us-central1

devai review code -c ~/github/genai-for-developers/sample-
app/src/main/java/anthos/samples/bankofanthos/balancereader
```

輸出內容範例：

```
(venv) student_00_19a997c157f8@cloudshell:~/genai-for-developers/devai-cli (qwiklabs-
gcp-02-71a9948ae110)$ devai review code -c ../sample-
app/src/main/java/anthos/samples/bankofanthos/balancereader
/home/student_00_19a997c157f8/genai-for-developers/devai-cli/venv/lib/python3.9/site-
packages/langchain_core/_api/deprecation.py:117: LangChainDeprecationWarning: The
function `initialize_agent` was deprecated in LangChain 0.1.0 and will be removed in
0.2.0. Use new agent constructor methods like create_react_agent, create_json_agent,
create_structured_chat_agent, etc. instead.
    warn_deprecated(
Response from Model: ```java
// Class: TransactionRepository
// Method: findBalance

// Efficiency
- Consider using a native SQL query to improve performance for complex database
operations.
- Use prepared statements to avoid SQL injection vulnerabilities.

// Best Practices
- Return a Optional<Long> instead of null to handle the case when no balance is found
for the given account.
```

```
/home/student_00_19a997c157f8/genai-for-developers/devai-cli/venv/lib/python3.9/site-
packages/langchain_core/_api/deprecation.py:117: LangChainDeprecationWarning: The function __call__ was
deprecated in LangChain 0.1.0 and will be removed in 0.2.0. Use invoke instead. warn_deprecated(
```

Entering new AgentExecutor chain... 想法：問題中已提供說明，因此無需思考。動作：

```
{
  "action": "create_issue",
  "action_input": {
    "description": "Class: TransactionRepository\nMethod: findBalance\n\nEfficiency\n-
Consider using a native SQL query to improve performance for complex database operations.\n-
Use prepared statements to avoid SQL injection vulnerabilities.\n\nBest Practices\n- Return a
Optional<Long> instead of null to handle the case when no balance is found for the given
account."
  }
}
```

已建立新問題，金鑰為：CYMEATS-117

觀察：已建立新問題，金鑰為 CYMEATS-117。想法：最終答案：CYMEATS-117

完成鏈結。

Open your JIRA project in the browser and review the created issue.

Sample JIRA issue view.

Open [LangSmith portal](https://smith.langchain.com/) and review LLM trace for JIRA issue creation call.

Sample LangSmith LLM trace.

Import GitHub repo to GitLab repo

Go to https://gitlab.com/projects/new and select "`Import project`" / "`Repository by URL`" option:

Git repository url:

<https://github.com/GoogleCloudPlatform/genai-for-developers.git>

Or

Your personal GitHub project that you created earlier in this lab.

Under Project URL - select your GitLab userid

Set Visibility to `Public`.

Click - "`Create Project`" to start the import process.

If you see an error about invalid GitHub Repository URL, [create a new GitHub token] (<https://github.com/settings/tokens>)(fine-grained) with Public repositories read-only access, and retry import again providing your GitHub userid and token.

Clone GitLab repo and setup SSH key

Return to Google Cloud Shell terminal and set up a new SSH key.

Update your email before running the commands. Hit enter multiple times to accept defaults.

```
ssh-keygen -t ed25519 -C "your-email-address"
```

```
eval "$(ssh-agent -s)" ssh-add ~/.ssh/id_ed25519
```

```
cat ~/.ssh/id_ed25519.pub
```

Add a public key to your GitLab account.

Open <https://gitlab.com/-/profile/keys> and click "Add new key".

For the key value copy/paste the output of the last command.

Go back to the terminal and clone the repository.

```
cd ~ mkdir gitlab cd gitlab
```

Replace with your GitLab userid and repository url that was just created.

```
```console
git clone git@gitlab.com:YOUR_GITLAB_USERID/genai-for-developers.git
```

變更目錄並開啟 `.gitlab-ci.yml` 檔案。

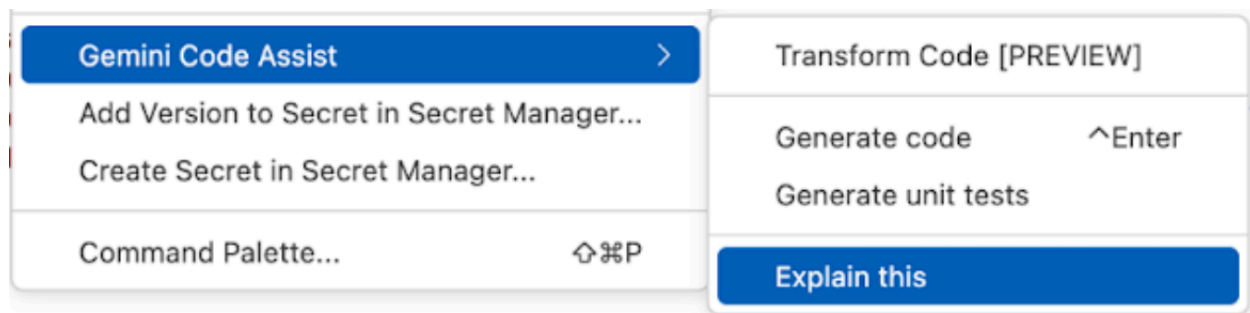
```
cd genai-for-developers

cloudshell edit .gitlab-ci.yml
```

如果先前未啟用 Gemini，請在 Cloud Shell 編輯器中啟用。



在 `.gitlab-ci.yml` 檔案的任意位置按一下滑鼠右鍵，然後選取「Gemini Code Assist > Explain this"」。



步驟 18 · 約 0 分鐘

## 18. GitLab 指令設定

開啟 [GitLab](#)，並在先前步驟建立的 GitLab 存放區中，於「[Settings](#) / [Access Tokens](#)」下方建立專案存取權杖。

複製並儲存存取權杖值，以供後續步驟使用。

使用下列詳細資料：

- 權杖名稱：`devai-cli-qwiklabs`
- 角色：`Maintainer`
- 範圍：`api`

### Project Access Tokens

Generate project access tokens scoped to this project for your application HTTP(S). [Learn more.](#)

**Active project access tokens** 0

#### Add a project access token

**Token name**

For example, the application using the token or the purpose of the token

**Expiration date**

2024-08-23 

**Select a role**

Maintainer 

**Select scopes**

Scopes set the permission levels granted to the token. [Learn more.](#)

☒ `api`  
Grants complete read and write access to the scoped project API, i

設定 GitLab 整合功能所需的環境變數。

這項指令需要您更新 GitLab 存取權杖。

```
export GITLAB_PERSONAL_ACCESS_TOKEN=gitlab-access-token
```

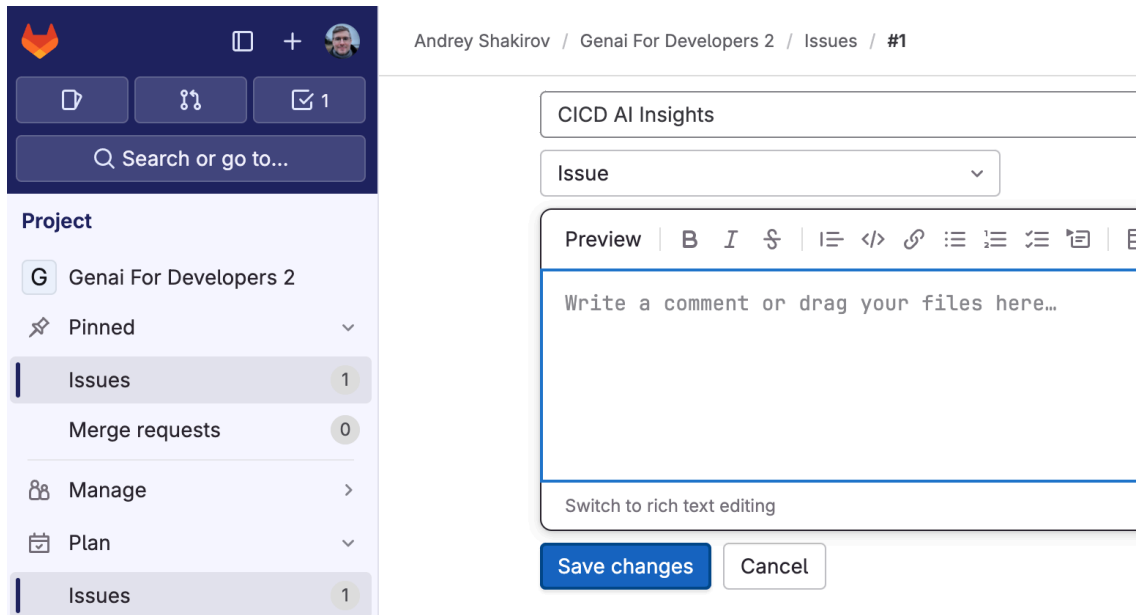
這項指令需要您更新 GitLab 使用者 ID 和存放區名稱。

```
export GITLAB_REPOSITORY="USERID/REPOSITORY"
```

設定其餘環境變數：

```
export GITLAB_URL="https://gitlab.com"
export GITLAB_BRANCH="devai"
export GITLAB_BASE_BRANCH="main"
```

開啟 [GitLab](#) 網站，並在專案中建立新 GitLab 問題，標題為「`CICD AI Insights`」。



您也可以使用下列 curl 指令。您需要 GitLab 專案 ID，可以在「Settings / General」部分下方查詢。

```
export GITLAB_PROJECT_ID=56390153 # replace

curl --request POST \
 --header "PRIVATE-TOKEN: $GITLAB_PERSONAL_ACCESS_TOKEN" \
 --header "Content-Type: application/json" \
 --data '{"title":"CICD AI Insights"}' \
 https://gitlab.com/api/v4/projects/$GITLAB_PROJECT_ID/issues
```

返回 Cloud Shell 並開啟 `review.py` 檔案：

```
cloudshell edit ~/gitlab/genai-for-developers/devai-cli/src/devai/commands/review.py
```

找出並取消註解下列程式碼

匯入 GitLab 指令的行

```
from devai.commands.gitlab import create_gitlab_issue_comment
```

在 GitLab 問題中留言的方法

```
create_gitlab_issue_comment(response.text)
```



## 19. DevAI CLI 開發

因為您已切換至 GitLab 存放區/目錄。你必須重新執行下列設定步驟。

在終端機中設定 Python 虛擬環境、安裝必要條件，然後執行範例指令。

```
export PROJECT_ID=$(gcloud config get-value project)
export LOCATION=us-central1

cd ~/gitlab/genai-for-developers/devai-cli
python3 -m venv venv
. venv/bin/activate
pip3 install -r src/requirements.txt
pip3 install --editable ./src
devai echo
```

您可以確認 CLI 的位置，這次應該位於 GitLab 資料夾下方。

```
which devai
```

在終端機中重新執行程式碼審查指令：

```
devai review code -c ~/gitlab/genai-for-developers/sample-
app/src/main/java/anthos/samples/bankofanthos/balancereader
```

輸出內容範例 (部分區段已縮短)：

```
(venv) student_00_19a997c157f8@cloudshell:~/genai-for-developers/devai-cli (qwiklabs-gcp-02-71a9948ae110)$ devai review code -c ../sample-app/src/main/java/anthos/samples/bankofanthos/balancereader
```

```
.
.
Response from Model: **Class: Transaction**

Method: toString

Maintainability:

* The formatting of the string representation could be more clear and concise.
Consider using a dedicated method for formatting the amount, e.g., `formatAmount()`.
.
.
```

```
> Entering new AgentExecutor chain...
```

```
Thought: I need to first get the issue ID using the Get Issues tool, then I can
comment on the issue using the Comment on Issue tool.
```

```
Action: Get Issues
```

```
Action Input:
```

```
Observation: Found 1 issues:
```

```
[{'title': 'CICD AI Insights', 'number': 1}]
```

```
Thought:Thought: I found the issue ID, so now I can add the comment to the issue.
```

```
Action: Comment on Issue
```

```
Action Input: 1
```

```
Action: Get Issue
```

```
Action Input: 1
```

```
Observation: {"title": "CICD AI Insights", "body": "", "comments": "[{'body':
 '**Transaction.java**\\n\\n\\n**Class:** Transaction\\n\\n\\n* **Security:** Consider
using a custom date format like '\\\\\\"yyyy-MM-dd HH:mm:ss.SSS\\\\\\"' to handle
timestamps more robustly.\\n\\n\\n**JWTVerifierGenerator.java**\\n\\n\\n* .
.
```

```
Thought:Now I can use the Comment on Issue tool to add the comment to the issue.
```

```
Action: Comment on Issue
```

```
Action Input:
```

```
1
```

```
Class: Transaction
```

```
Method: toString
```

```
Maintainability:
```

```
.
.
```

Observation: Commented on issue 1

Thought:I have now completed the necessary actions and added the comment to the issue 'CICD AI Insights'.

Final Answer: Comment added to issue 'CICD AI Insights'

> Finished chain.

開啟 [GitLab](#) 網站，查看更新後的問題。

The screenshot displays the GitLab web interface. On the left is a sidebar with navigation options: Project, Pinned, Issues (1), Merge requests (0), Manage, Plan, Issue boards, Milestones, Wiki, Code, Build, Secure, Deploy, Operate, Monitor, Analyze, and Settings. The main content area shows a code review for the 'Bank of Anthos Balance Reader' by user 'devai-cli-qwiklabs'. The review includes a summary of the code's purpose and detailed feedback on transaction processing logic, error logging, JWT verification, and database synchronization. The feedback is organized into sections for each file: 'BalanceReaderController.java' and 'LedgerReader.java'. Each section lists specific observations, potential issues, and suggestions for improvement.

Andrey Shakirov / Genai For Developers 2 / Issues / #1

Open CICD AI Insights

devai-cli-qwiklabs @project\_60301622\_bot\_c31ccc37e91c347242a0394fed4b6fc2 · just now (Maintainer)

### Code Review: Bank of Anthos Balance Reader

This code implements a balance reader service for the Bank of Anthos application. Here's a review based on the provided code snippets:

#### BalanceReaderController.java

- Transaction Processing Logic (Lines 77-95):**
  - The transaction processing logic within the `startWithCallback` method appears to directly manipulate the cache (`this.cache.put`) based on incoming transactions.
  - Potential Issue (Medium):** Directly manipulating the cache inside the callback might introduce inconsistencies, especially in a distributed environment.
  - Suggestion:** Consider using a dedicated mechanism like a message queue or a distributed cache to handle transaction updates and maintain cache consistency.
- Error Logging (Line 146):**
  - The catch block for `ExecutionException` and `UncheckedExecutionException` logs a generic "Cache error" message.
  - Potential Issue (Low):** The generic error message lacks context, making debugging difficult.
  - Suggestion:** Include the exception message or stack trace in the log for better troubleshooting.
- JWT Verification (Lines 129-141):**
  - The `getBalance` method handles JWT verification and authorization. While the logic is correct, extracting it into a separate authentication/authorization layer could enhance code clarity and maintainability.
  - Suggestion (Medium):** Consider introducing an `@PreAuthorize` annotation (if using Spring Security) or a separate interceptor to handle JWT verification and authorization, keeping the controller logic focused on balance retrieval.

#### LedgerReader.java

- Database Synchronization (Lines 109-113):**
  - The code suspends transaction processing if the remote database transaction ID is less than the local one, indicating synchronization issues.
  - Potential Issue (High):** The current approach simply stops processing, which could lead to data loss or inconsistencies.

在 [LangSmith](#) 中查看 LLM 追蹤記錄。

LLM 追蹤記錄範例。

## TRACE

Collapse Stats Filter Show All 

AgentExecutor   36.43s  5,951 zero-shot-react-description

```

graph TD
 LLMChain1[LLMChain 19.13s] --> ChatVertexAI1[ChatVertexAI gemini-1.5-pro 19.13s]
 ChatVertexAI1 --> GetIssues[Get Issues 0.18s]
 GetIssues --> LLMChain2[LLMChain 15.05s]
 LLMChain2 --> ChatVertexAI2[ChatVertexAI gemini-1.5-pro 15.04s]
 ChatVertexAI2 --> CommentOnIssue[Comment on Issue 0.76s]
 CommentOnIssue --> LLMChain3[LLMChain 1.29s]
 LLMChain3 --> ChatVertexAI3[ChatVertexAI gemini-1.5-pro 1.29s]

```

The diagram illustrates a sequence of tasks and their associated AI models and completion times. The tasks are connected by lines, indicating a sequential flow.

- LLMChain 19.13s
- ChatVertexAI gemini-1.5-pro 19.13s
- Get Issues 0.18s
- LLMChain 15.05s
- ChatVertexAI gemini-1.5-pro 15.04s
- Comment on Issue 0.76s
- LLMChain 1.29s
- ChatVertexAI gemini-1.5-pro 1.29s


**AgentExecutor**

[Run](#)   [Feedback](#)   [Metadata](#)

Input 

```

1 input: |
2 You need to do two tasks only.
3 First task: Get GitLab issue with title
4 'CiCD AI Insights'.
5 Second task: add content below as a
6 comment to the issue you found in first task:
7
8 """ Code Review: Bank of Anthos Balance
9 Reader\n\nThis code implements a balance reader
10 service for the Bank of Anthos application.
11 Here's a review based on the provided code
12 snippets:\n\n**BalanceReaderController.java**\n\n
13 * **Transaction Processing Logic (Lines 77-
14 95):** \n * The transaction processing
15 logic within the `startWithCallback` method
16 appears to directly manipulate the cache
17 (`this.cache.put`) based on incoming
18 transactions.\n * **Potential Issue
19 (Medium):** Directly manipulating the cache
20 inside the callback might introduce
21 inconsistencies, especially in a distributed
22 environment. \n * **Suggestion:** Consider

```

YAML Output 

```
1 output: I have successfully completed both
 tasks.
2 ✓ intermediate steps:
```

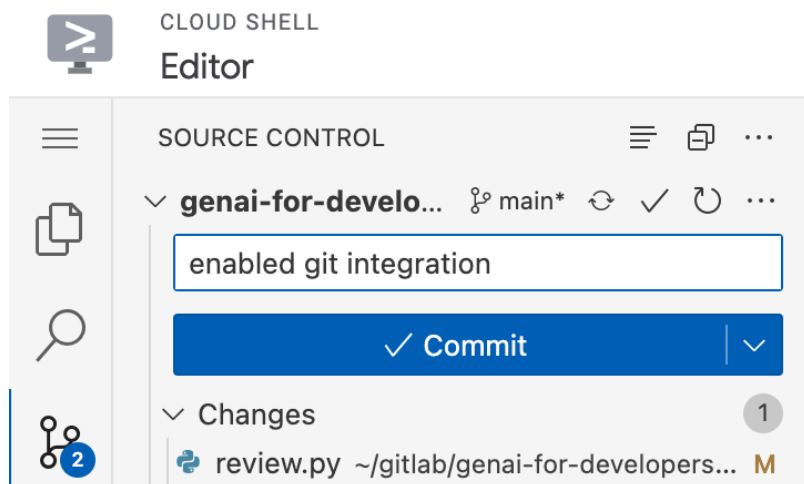
步驟 20 · 約 0 分鐘

## 20. 將變更推送至 GitLab 存放區

返回 Google Cloud Shell 編輯器。

切換至「Source Control」分頁。

暫存、提交並推送所做的變更，以更新 `review.py` 檔案。



步驟 21 · 約 0 分鐘

## 21. GitLab CI/CD 設定

接下來，您要啟用 GitLab CI/CD 管道，以便在變更推送至存放區時執行程式碼審查。

開啟 [GitLab](#) 網站，然後前往「Settings / CI/CD」部分。

展開 **Variables** 部分，然後按一下「Add variable」。

新增變數時，請務必取消勾選所有核取方塊。範例：

Add variable

Type

Variable (default)

Environments

All (default)

Visibility

☒ Visible

Can be seen in job logs.

☐ Masked

Masked in job logs but value can be revealed in CI/CD settings. Requires values to meet regular expressions requirements.

Flags

☐ Protect variable

Export variable to pipelines running on protected branches and tags only.

☐ Expand variable reference

\$ will be treated as the start of a reference to another variable.

Description (optional)

The description of the variable's value or usage.

Key

PROJECT\_ID

You can use CI/CD variables with the same name in different places, but the variables might overwrite each other. [What is the order of precedence for variables?](#)

Value

qwiklabs-gcp-00-050c89e09327

Variable value will be evaluated as raw string.

Add variable

Cancel

使用筆記 (您在其中保留所有環境變數)，為 JIRA、GitLab 和 LangSmith 新增環境變數。

```
PROJECT_ID=qwiklabs-project-id
LOCATION=us-central1
GOOGLE_CLOUD_CREDENTIALS - cat ~/vertex-client-key.json

LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT="https://api.smith.langchain.com"
LANGCHAIN_API_KEY=your-service-api-key

JIRA_API_TOKEN=your-token
JIRA_USERNAME="email that you used to register with JIRA"
JIRA_INSTANCE_URL="https://YOUR-PROJECT.atlassian.net"
JIRA_PROJECT_KEY="JIRA project key"
JIRA_CLOUD=true

GITLAB_PERSONAL_ACCESS_TOKEN=your-gitlab-token
GITLAB_URL="https://gitlab.com"
GITLAB_REPOSITORY="USERID/REPOSITORY"
GITLAB_BRANCH="devai"
GITLAB_BASE_BRANCH="main"
```

針對 `GOOGLE_CLOUD_CREDENTIALS` 變數值，請使用上方章節中建立的服務帳戶金鑰。

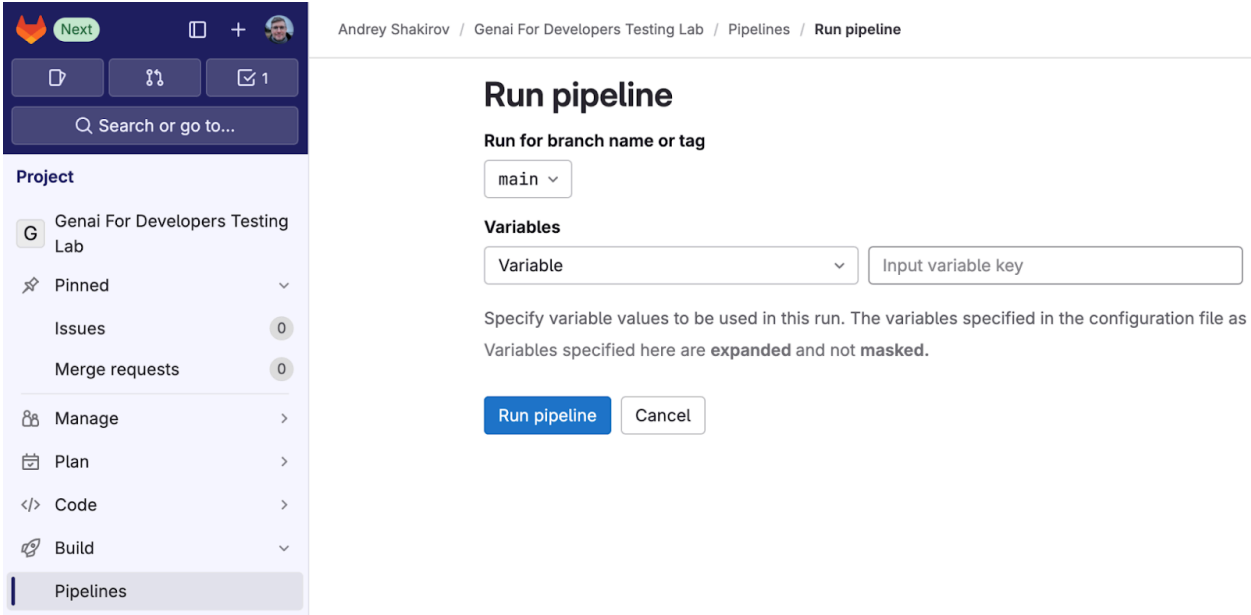
```
cat ~/vertex-client-key.json
```

CI/CD 變數檢視畫面：

CI/CD Variables </> 3				Reveal values	Add variable
Key ↑	Value	Environments	Actions		
GOOGLE_CLOUD_CREDENTIALS	*****	All (default)			
LOCATION	*****	All (default)			
PROJECT_ID	*****	All (default)			

## 22. 執行 GitLab CI/CD 管道

在 GitLab UI 中開啟「Build / Pipelines」，然後按一下「Run Pipeline」。

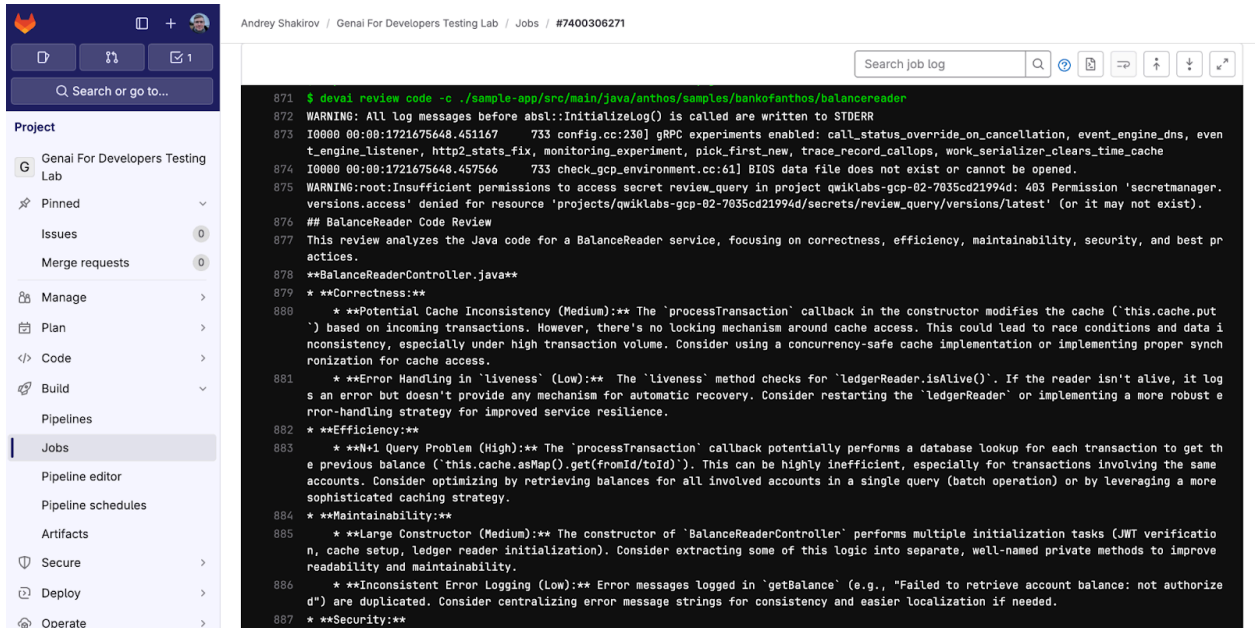




步驟 23 · 約 0 分鐘

## 23. 查看 GitLab 管道輸出內容

在 GitLab UI 中開啟「Build / Jobs」，並檢查管道輸出內容。



開啟 [GitLab](#) 網站，查看「[CICD Insights](#)」問題的更新註解。

## 停用 GitLab 工作流程執行作業

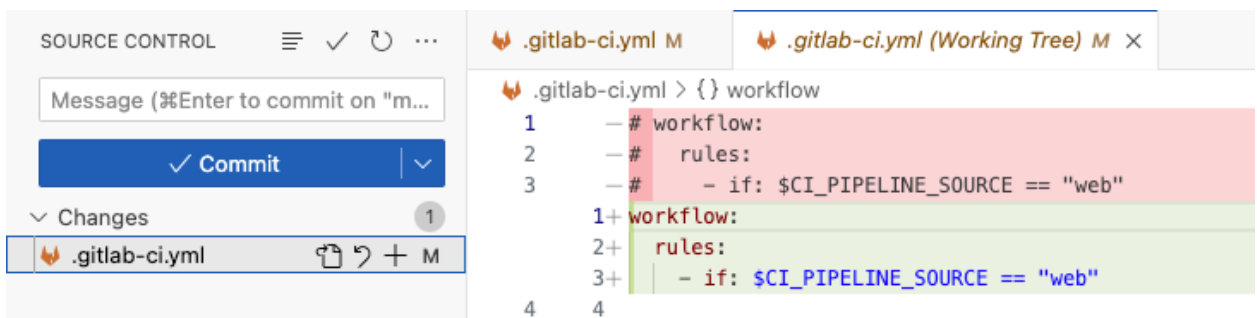
返回 Google Cloud Shell 編輯器。取消註解這些行，即可在程式碼推送事件中停用 GitLab 工作流程執行作業。您還是可以透過 UI 視需要執行工作流程。

```
workflow:
rules:
- if: $CI_PIPELINE_SOURCE == "web"
```

開啟專案根目錄中的 `.gitlab-ci.yml`，並取消註解下列幾行：

```
cloudshell edit ~/gitlab/genai-for-developers/.gitlab-ci.yml
```

切換至「[Source Control](#)」分頁，暫存、提交並推送這項變更。



## 24. CircleCI 整合

### 什麼是 CircleCI ？

CircleCI 是雲端式 CI/CD 平台，可讓團隊自動執行軟體開發和部署程序。與 GitHub、Bitbucket 和 GitLab 等版本管控系統整合，讓團隊能執行自動化測試和建構作業，即時驗證程式碼變更。在持續推送軟體更新方面，CircleCI 可自動將軟體部署至各種雲端環境，例如 AWS、Google Cloud 和 Azure。

### 設定

開啟 [CircleCI](#) 網站並建立新專案。為存放區選取「GitLab」/「Cloud」。

授予 CircleCI 存取 GitLab 帳戶的權限。

在「最快」選項下方，選取 `main` 分支版本。CircleCI 可能會偵測到現有的設定檔，並略過這個步驟。

## Create New Project

Connect to your GitLab.com repository and start running pipelines.

Connect to

 GitLab.com ▼

Connected to [gitrey](#) ✓

Repository

Genai For Developers gitrey ▼

✓ `.circleci/config.yml` found

We will listen for changes and setup a configuration file in this repository. Not seeing a repository? Check that you have the correct level of permissions on the repository.

Project Name

Genai For Developers

Enter 3-40 letters, numbers, or - \_ . ! & + [ ]. Project name cannot begin with a number or special character.

Create Project Cancel

建立專案後，按一下「Project Settings」/「Environment Variables」部分。

 **Genai For Developers** Project Settings

## Filters

 Everyone's Pipelines ▾ Genai For Developers ▾Auto-expand ☐ Select a Branch ▾ All days ▾ ▾**Commit a change to start running pipelines**

Now that your [configuration](#) and [triggers](#) are set up, commit a change to your code source to trigger your first pipeline. You'll see it here once you do.

新增目前為止使用的所有環境變數。

**Project Settings**

Genai For Developers

**Overview**

People

Groups

Configuration

Triggers

Advanced

Environment Variables

以下是新增環境變數的範例清單。

```
PROJECT_ID=qwiklabs-project-id
LOCATION=us-central1
GOOGLE_CLOUD_CREDENTIALS - cat ~/vertex-client-key.json
```

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT="https://api.smith.langchain.com"
LANGCHAIN_API_KEY=your-service-api-key
```

```
JIRA_API_TOKEN=your-token
JIRA_USERNAME="email that you used to register with JIRA"
JIRA_INSTANCE_URL="https://YOUR-PROJECT.atlassian.net"
JIRA_PROJECT_KEY="JIRA project key"
JIRA_CLOUD=true
```

```
GITLAB_PERSONAL_ACCESS_TOKEN=your-gitlab-token
GITLAB_URL="https://gitlab.com"
GITLAB_REPOSITORY="USERID/REPOSITORY"
GITLAB_BRANCH="devai"
GITLAB_BASE_BRANCH="main"
```

---

步驟 25 · 約 0 分鐘

## 25. 啟用 JIRA 和 GitLab 方法

開啟 Google Cloud Shell 編輯器，然後變更 `review.py` 檔案。

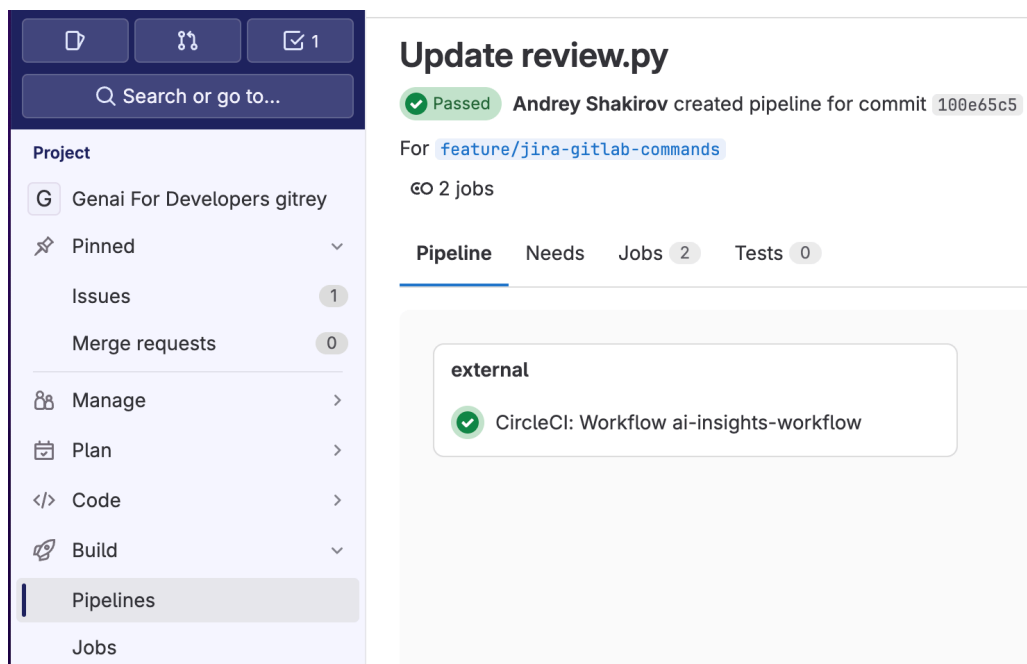
找出並取消註解下方的程式行。

```
from devai.commands.jira import create_jira_issue

create_jira_issue("Performance Review Results", response.text)
create_gitlab_issue_comment(response.text)
.
.
.
create_jira_issue("Security Review Results", response.text)
create_gitlab_issue_comment(response.text)
```

切換至「Source Control」分頁，暫存、提交並推送這項變更。

開啟 [GitLab](#) 網站，然後前往「Build」/「Pipelines」。



點選 CircleCI 的連結，即可查看工作流程。

ai-insights-workflow

Success

Duration / Finished	Branch	Commit	Author & Message
🕒 1m 5s / 58m ago	🔗 feature/jira-gitlab-commands	🔑 c1f69c3	👤 Add new file

✓ ai-insights-security-review54s

✓ ai-insights-performance... 42s

✓ ai-insights-code-review 1m 3s

查看存放區中 GitLab 問題的留言。

🔍 Search or go to...

Project

Genai For Developers 2

Pinned

Issues1

Merge requests0

Manage

Plan

Issues1

Issue boards

Milestones

Wiki

Code

Build

Secure

Deploy

Operate

Monitor

Analyze

Settings

Andrey Shakirov / Genai For Developers 2 / Issues / #1

OpenCICD AI Insights

devai-cli-qwiklabs @project\_60301622\_bot\_c31ccc37e91c347242a0394fed4b6fc2 · 15 minutes agoMaintainer

### Java Performance Review: Bank of Anthos BalanceReader

Here's a performance review of the provided Java code, focusing on the `BalanceReader` service.

File: `./sample-app/src/main/java/anthos/samples/bankofanthos/balancereader/BalanceCache.java`

Function: `initializeCache`

- **Potential Bottleneck:** The `load` method in your `CacheLoader` fetches the balance from the database (`dbRepo.findBalance`) every time a cache miss occurs. This could be a performance bottleneck if the database latency is high or if you have frequent cache misses.
  - **Suggestion (High impact):**
    - **Investigate Caching at the Database Layer:** Explore caching mechanisms within your database itself to optimize `findBalance` query performance. This might involve database-specific caching configurations or using tools like Redis for a dedicated caching layer.
    - **Cache Pre-warming:** Consider pre-populating the cache with frequently accessed account balances during application startup or at scheduled intervals. This can reduce initial cache misses.
  - **Consider (Medium impact):**
    - **Batching Database Requests:** If multiple balance lookups are often needed concurrently, consider fetching balances for multiple accounts in a single database query rather than individual lookups.
- **Clarification:** What is the expected hit ratio for your cache? Understanding how often you expect to find balances in the cache versus going to the database is crucial for evaluating the effectiveness of your caching strategy.

File: `./sample-app/src/main/java/anthos/samples/bankofanthos/balancereader/BalanceReaderController.java`

- **Function: `getBalance`**
  - **Potential Bottleneck:** The `cache.get(accountId)` call, while using a cache, still has a potential performance impact if there are frequent cache misses. The `load` method in your `BalanceCache` would then be invoked, which makes a database call.
  - **Suggestion:** Refer to the suggestions provided for `initializeCache` in `BalanceCache.java`. Optimizing your cache hit ratio and database access patterns will directly benefit the performance of `getBalance`.
  - **Error Handling:** You're catching broad exceptions (`ExecutionException`, `UncheckedExecutionException`). While this isn't necessarily a performance issue, it's good practice to catch more specific exceptions to handle different error scenarios gracefully and potentially provide more informative error messages to the client.
- **General Observation:** The `LedgerReader.startWithCallback` method is called in the constructor. This means that the background thread processing transactions starts immediately when a `BalanceReaderController` instance is created.

Search or go to...

Project

Genai For Developers 2

Pinned

Issues1

Merge requests0

Manage

Plan

Issues1

Issue boards

Milestones

Wiki

Code

Build

Secure

Deploy

Operate

Monitor

Analyze

Settings

Help

Open

CICD AI Insights

devai-cli-qwiklabs @project\_60301622\_bot\_c31ccc37e91c347242a0394fed4b6fc2 · 17 minutes ago

Maintainer

BalanceReader Java Code Security Review

This code review focuses on the Java code snippets you provided for the BalanceReader application. Below is a breakdown of potential security vulnerabilities and recommendations, organized by file and function/method.

Overall Observations

Positive:

 The codebase demonstrates good security practices in some areas, like using a JWT verifier for authentication and attempting to sanitize input by only accepting "Bearer " tokens.

Concerns:

 There are areas where security best practices are not fully implemented, particularly around input validation and potential SQL injection vulnerabilities.

File: `./sample-app/src/main/java/anthos/samples/bankofanthos/balancereader/BalanceCache.java`

No significant vulnerabilities identified.

File: `./sample-app/src/main/java/anthos/samples/bankofanthos/balancereader/BalanceReaderApplication.java`

No significant vulnerabilities identified.

Recommendation (Informational):

 Avoid logging sensitive information like database connection strings or environment variables in clear text. Use a secure logging mechanism to mask or encrypt such data.

File: `./sample-app/src/main/java/anthos/samples/bankofanthos/balancereader/BalanceReaderController.java`

Issue: Potential SQL Injection Vulnerability (Severity: Critical)

Location:

`getBalance` method, indirect use of `accountId` in `cache.get(accountId)`.

Impact:

 The `accountId` obtained from the URL path parameter is directly used as a key for the cache. If the underlying cache implementation uses this key to construct SQL queries without proper sanitization, it could lead to SQL injection.

Recommendation:

Validate and Sanitize Input:

 Never trust user-supplied input. Validate that the `accountId` only contains alphanumeric characters or a limited set of allowed characters.

Parameterized Queries (Best Practice):

 Use parameterized queries or prepared statements when interacting with your database in the `TransactionRepository` to prevent SQL injection.

Issue: Potential Information Leakage via Error Messages (Severity: Medium)

Location:

`getBalance` method

Impact:

 Detailed error messages like "JWTVerificationException" might reveal sensitive implementation details to an attacker.

查看在 JIRA 專案中建立的新問題。

ProjectsFiltersDashboardsTeamsAppsCreate

Add epic

CYMEATS-238

+ Add

@ Apps

Description

1.

a. BalanceReader Java Code Security Review

This code review focuses on the Java code snippets you provided for the BalanceReader application. Below is a breakdown of potential security vulnerabilities and recommendations, organized by file and function/method.

\*Overall Observations\*

\*Positive:\*

 The codebase demonstrates good security practices in some areas, like using a JWT verifier for authentication and attempting to sanitize input by only accepting "Bearer " tokens.

\*Concerns:\*

 There are areas where security best practices are not fully implemented, particularly around input validation and potential SQL injection vulnerabilities.

\*File: `./sample-app/src/main/java/anthos/samples/bankofanthos/balancereader/BalanceCache.java` \*

No significant vulnerabilities identified.\*

\*File: `./sample-app/src/main/java/anthos/samples/bankofanthos/balancereader/BalanceReaderApplication.java` \*

No significant vulnerabilities identified.\*

\*Recommendation (Informational):\*

 Avoid logging sensitive information like database connection strings or environment variables in clear text. Use a secure logging mechanism to mask or encrypt such data.

\*File: `./sample-app/src/main/java/anthos/samples/bankofanthos/balancereader/BalanceReaderController.java` \*

Issue:\*

 Potential SQL Injection Vulnerability (Severity: \*Critical\*)

Location:\*

`getBalance` method, indirect use of `accountId` in `cache.get(accountId)`.

Add a comment...

Pro tip: press **M** to comment





步驟 26 · 約 0 分鐘

## 26. 恭喜！

恭喜，您已完成本實驗室！

### 涵蓋內容：

- 在 GitHub、GitLab 和 CircleCI 中新增生成式 AI 程式碼審查自動化步驟。
- LangChain ReAct 代理程式，可自動執行在 GitLab 問題加上註解，以及開啟 JIRA 支援單等工作。

### 後續步驟：

- 更多實務課程即將推出！

## 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

## 刪除專案

如要避免付費，最簡單的方法就是刪除您為了本教學課程所建立的專案。

©2024 Google LLC 保留所有權利。Google 和 Google 標誌是 Google LLC 的商標，所有其他公司和產品名稱可能是其關聯公司的商標。

---